

BÁO CÁO THỰC HÀNH

Môn học: Bảo mật web và ứng dụng

Tên chủ đề: Tổng quan các lỗ hổng web thường gặp

GVHD: Ngô Khánh Khoa

Nhóm: 11

1. THÔNG TIN CHUNG:

Lớp: NT213.P11.ANTT

STT	Họ và tên	MSSV	Email
28	Nguyễn Đăng Quỳnh Như	22521050	22521050@gm.uit.edu.vn
33	Phạm Minh Tân	22521310	22521310@gm.uit.edu.vn
31	Lê Minh Quân	22521181	22521181@gm.uit.edu.vn
32	Đặng Đức Tài	22521270	22521270@gm.uit.edu.vn

2. NỘI DUNG THỰC HIỆN:¹

STT	Nội dung	Tình trạng	Trang
1	Bài tập chính	100%	3-22
2	Bài tập thêm	100%	22-51
Điểm tự đánh giá			9.5/10

Phần bên dưới của báo cáo này là tài liệu báo cáo chi tiết của nhóm thực hiện.

Link video top 5 OSWAP: [LINK TOP 5 OSWAP](#)

Link video bài tập thêm:

1-13: [Bài Tập Thêm](#)

¹ Ghi nội dung công việc, các kịch bản trong bài Thực hành

Mục Lục

Bài tập 1: Broken Access Control	3
Bài tập 2: Cryptographic Failures	4
Bài tập 3: Injection	6
Bài tập 4: Insecure Design	9
Bài tập 5: Security Misconfiguration	12
Bài tập luyện tập	15
http://localhost:8000/broken_access_lab_2	16
http://localhost:8000/broken_access_lab_3	17
http://localhost:8000/cryptographic_failure/lab2	20
http://localhost:8000/cryptographic_failure/lab3	22
http://localhost:8000/cmd_lab	24
http://localhost:8000/ssti/lab	25
http://localhost:8000/data_exp_lab	26
http://localhost:8000/sec_mis_lab3	28
https://portswigger.net/web-security/sql-injection/examining-the-database/lab-listing-database-contents-oracle	30
https://portswigger.net/web-security/file-path-traversal/lab-absolute-path-bypass	34
https://portswigger.net/web-security/access-control/lab-multi-step-process-with-no-access-control-on-one-step	37
https://portswigger.net/web-security/logic-flaws/examples/lab-logic-flaws-infinite-money	42
https://portswigger.net/web-security/prototype-pollution/client-side/browser-apis/lab-prototype-pollution-client-side-prototype-pollution-via-browser-apis	45

BÁO CÁO CHI TIẾT

PHẦN I: BÀI TẬP CƠ BẢN

Bài tập 1: Broken Access Control

Tiêu đề: Broken Access Control trong chức năng kiểm soát quyền truy cập của trang Lab 1

Mô tả lỗi hỏng: Lỗi hỏng này là một lỗi hỏng kiểm soát truy cập bị phá vỡ (Broken Access Control) trong trang web thực hành Lab 1. Khi người dùng đăng nhập bằng tài khoản thường và điều chỉnh cookie admin=1, hệ thống không xác thực quyền hợp lệ mà chấp nhận thay đổi này, dẫn đến việc người dùng thường có thể truy cập vào các trang quản trị mà không có quyền.

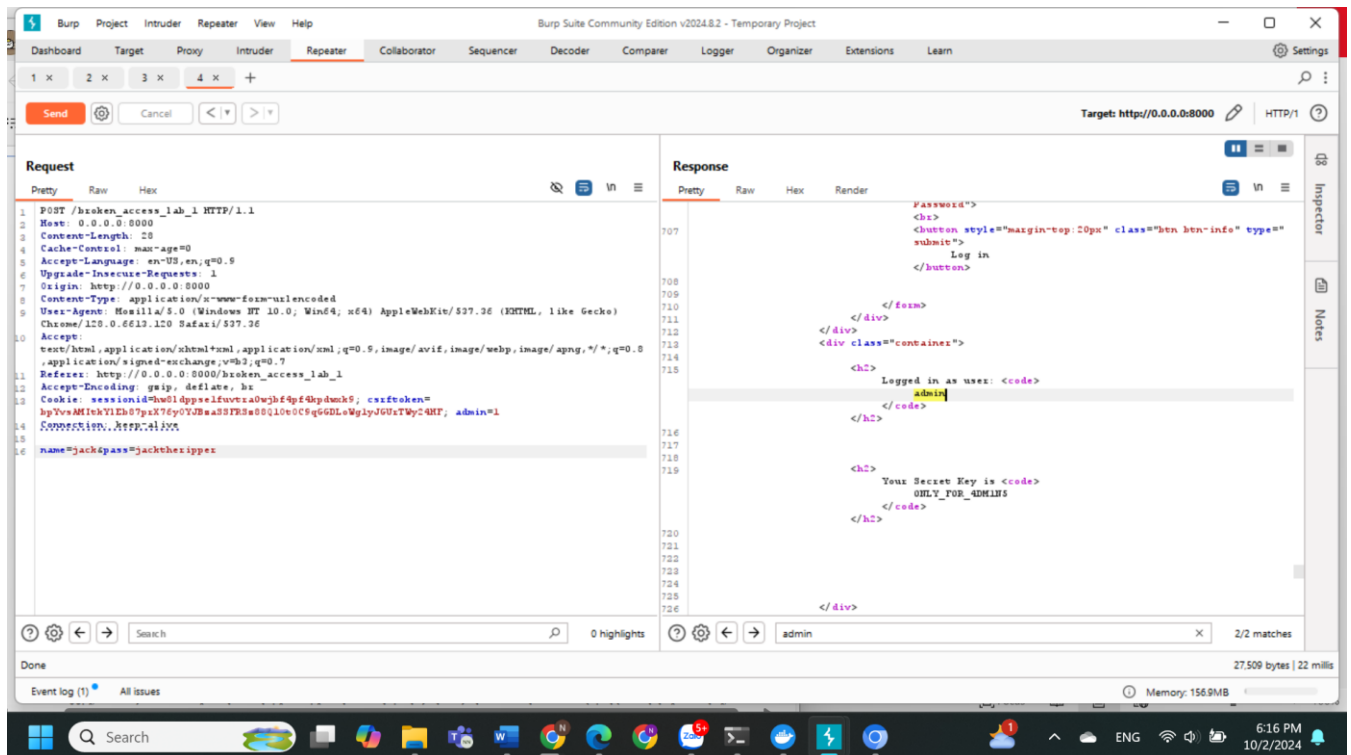
Tóm tắt:

Trang Lab 1 đã gặp phải một lỗi hỏng Broken Access Control do không kiểm soát chặt chẽ các thuộc tính của cookie. Người dùng có thể dễ dàng nâng cao đặc quyền bằng cách thay đổi giá trị admin trong cookie mà không cần xác thực lại. Bằng cách thay đổi admin=0 thành admin=1, người dùng Jack có thể truy cập vào thông tin nhạy cảm và lấy được khóa bí mật của admin.

Cách 1: sử dụng intercept on

Các bước để thực hiện lại và bằng chứng:

Đăng nhập vào trang web với tài khoản và mật khẩu được cung cấp của user Jack:jacktheripper, dùng Burpsuite để chuyển gói tin đã bắt được đến repeater. Tại đây sửa giá trị cookie admin: admin = 1. Sau đó ta thực hiện Send gói tin và thu được kết quả:



Tài liệu hỗ trợ và tham khảo:

- Sử dụng BurpSuite để kiểm tra và chỉnh sửa gói tin.
- Tham khảo tài liệu CORS tại: [PortSwigger - CORS](#)
- Tham khảo nguyên tắc đặc quyền tối thiểu: Principle of Least Privilege

Mức độ ảnh hưởng của lỗ hổng:

Lỗ hổng này cho phép bất kỳ người dùng nào (không cần quyền admin) có thể truy cập và kiểm soát trái phép các tài khoản và thông tin nhạy cảm trên trang web. Từ đó, họ có thể thực hiện các hành vi phá hoại như:

- Truy cập trái phép vào dữ liệu quản trị viên.
- Chỉnh sửa hoặc xóa dữ liệu trên hệ thống.
- Lấy cắp các thông tin nhạy cảm như khóa bí mật.

Khuyến cáo khắc phục:

1. **Xác thực lại quyền hạn người dùng** sau khi thay đổi bất kỳ thuộc tính đặc quyền nào như cookie hoặc token.
2. **Không sử dụng các trường nhạy cảm như admin trong cookie mà không mã hóa hoặc ràng buộc với người dùng.**
3. **Áp dụng kiểm tra quyền hạn chặt chẽ** trên tất cả các API và phương thức HTTP như POST, PUT, DELETE.
4. **Sử dụng các kỹ thuật mã hóa hoặc JWT hợp lệ** để bảo vệ giá trị token tránh việc giả mạo.

Bài tập 2: Cryptographic Failures

Tiêu đề: Lỗ hổng Mật mã MD5 dễ dàng bị tấn công brute-force

Mô tả lỗ hổng:

Lỗ hổng này là một lỗ hổng mật mã liên quan đến việc sử dụng thuật toán băm MD5, vốn đã lỗi thời và dễ dàng bị phá vỡ. MD5 không còn đảm bảo tính bảo mật vì nó có khả năng tạo ra các xung đột (collision) và dễ dàng bị tấn công brute-force, cho phép kẻ tấn công lấy lại thông tin gốc. Trong bài thực hành này, dữ liệu nhạy cảm của người dùng như mật khẩu được lưu trữ dưới dạng băm MD5, và khi kẻ tấn công thực hiện SQL Injection để trích xuất các bản ghi mật khẩu, họ có thể sử dụng các công cụ trực tuyến để giải mã chúng một cách dễ dàng.

Tóm tắt:

Trang web thực hành tại http://localhost:8000/cryptographic_failure/lab chứa thông tin đăng nhập của người dùng được lưu dưới dạng mã băm MD5. Bằng cách khai thác lỗ hổng SQL Injection, kẻ tấn công đã trích xuất được dữ liệu đăng nhập và sau đó sử dụng các công cụ bẻ khóa trực tuyến để giải mã các chuỗi MD5 thành mật khẩu gốc, từ đó có thể đăng nhập trái phép vào tài khoản của người dùng.

Các bước để thực hiện lại và bằng chứng:

- Bước 1:** Thực hiện tấn công SQL Injection để trích xuất bảng thông tin đăng nhập của người dùng từ cơ sở dữ liệu, ví dụ: `SELECT username, password FROM users;`
 - Kết quả trả về:
 - Username: john_doe
 - Password (MD5 hash): e99a18c428cb38d5f260853678922e03
- Bước 2:** Truy cập vào trang md5online.org và nhập chuỗi băm e99a18c428cb38d5f260853678922e03 để tìm mật khẩu tương ứng.
 - Kết quả tìm được: abc123.
- Bước 3:** Sử dụng mật khẩu abc123 để đăng nhập vào trang web tại `http://localhost:8000/cryptographic_failure/lab` với tài khoản john_doe.
 - Kết quả: Đăng nhập thành công với tư cách người dùng john_doe.

Tài liệu hỗ trợ và tham khảo:

- [MD5 Decrypt Online](#) - Công cụ giải mã trực tuyến giúp xác định các chuỗi băm MD5.
- Common Weakness Enumeration (CWE) - Các lỗ hổng bảo mật liên quan đến mật mã:
 - CWE-327:** Thuật toán mã hoá bị hỏng hoặc rủi ro.
 - CWE-328:** Hashing không an toàn.
 - CWE-259:** Sử dụng mật khẩu mã hoá cứng.

Mức độ ảnh hưởng của lỗ hổng:

Lỗ hổng này có mức độ nghiêm trọng cao vì MD5 đã không còn an toàn trong nhiều năm và có thể dễ dàng bị phá vỡ. Nếu mật khẩu người dùng được băm bằng MD5 và bị trích xuất, kẻ tấn công có thể lấy được mật khẩu gốc của người dùng trong vòng vài giây bằng các công cụ trực tuyến hoặc bảng tra cứu (rainbow tables). Điều này có thể dẫn đến:

- Đăng nhập trái phép vào tài khoản** và chiếm quyền điều khiển của người dùng hợp pháp.
- Tiết lộ thông tin cá nhân** của người dùng.
- Gây mất uy tín cho hệ thống** và làm mất niềm tin của người dùng đối với ứng dụng.

#Khuyến cáo khắc phục:

- Không sử dụng thuật toán MD5 hoặc SHA-1** để mã hóa mật khẩu hoặc các thông tin nhạy cảm. Thay vào đó, sử dụng các thuật toán hiện đại và an toàn hơn như bcrypt, scrypt hoặc Argon2.
- Sử dụng kỹ thuật băm kèm muối (salted hashing)** để ngăn chặn các cuộc tấn công dựa trên bảng tra cứu (rainbow tables).
- Áp dụng các chính sách bảo mật** để đảm bảo dữ liệu nhạy cảm được bảo vệ an toàn ở cả trạng thái nghỉ và trong quá trình truyền tải.
- Kiểm tra thường xuyên mã nguồn** và cơ sở dữ liệu để đảm bảo không có lỗ hổng mật mã nào tồn tại và tuân theo các tiêu chuẩn bảo mật như **OWASP Top 10** hoặc **PCI DSS**.

Bài tập 3: Injection

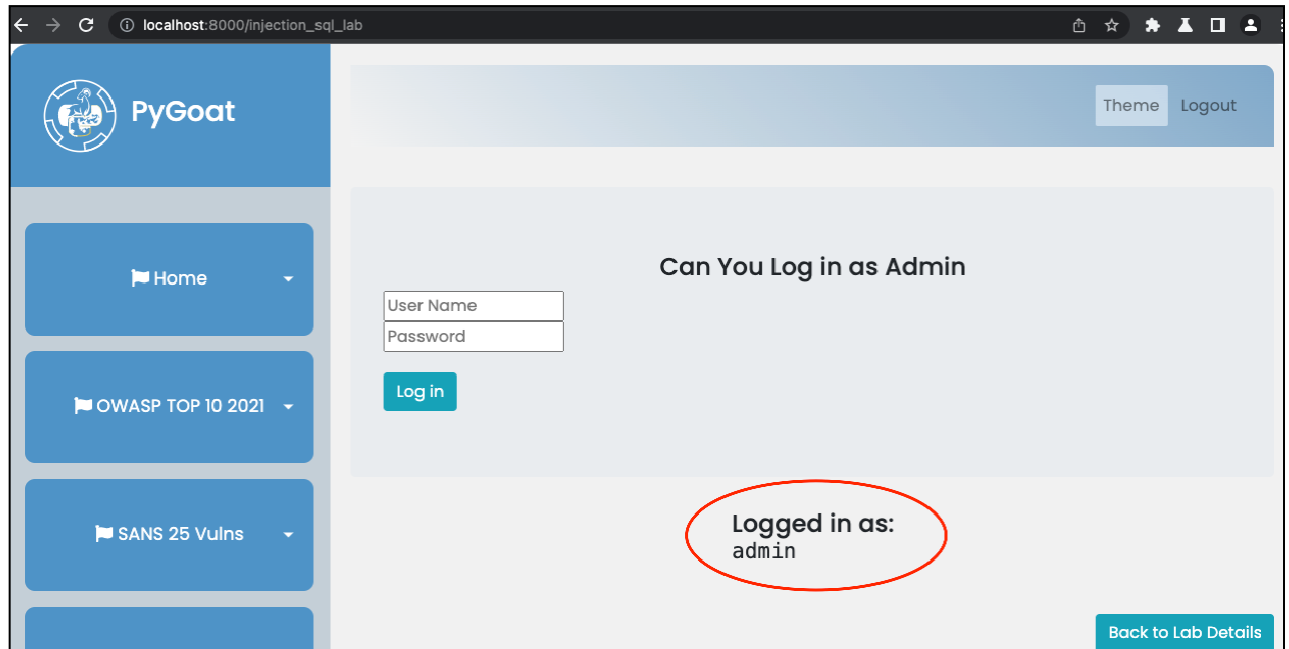
a. Mô tả

- Một ứng dụng có khả năng bị tấn công tiêm khi dữ liệu người dùng cung cấp không được xác thực hoặc lọc bởi ứng dụng.
- Tùy theo kiểu tiêm thì mã tấn công có thể khác nhau và tùy chỉnh theo cấu trúc của ứng dụng muốn tấn công, một số kiểu tiêm phổ biến như SQL injection, NoSQL injection, OS command injection, Object Relational Mapping(ORM) injection, LDAP injection, và Expression Language (EL) injection hoặc Object Graph Navigation Library (OGNL) injection.

b. Kịch bản tấn công thực hành

- Kịch bản thực hành này giúp cho học viên khai thác được lỗ hổng tiêm sql thông thường, gây ra khi thiếu xác thực đầu vào và truyền trực tiếp đầu vào vào câu truy vấn.

- Người dùng sẽ truy cập bài thực hành với trang đăng nhập được cung cấp. Người dùng có thể thử đăng nhập với tài khoản admin. Lỗi tiêm SQL có thể được nhận ra thông qua một vài thủ thuật như tiêm một kí tự ' vào bất kỳ trường nào. Nếu kết quả là một lỗi SQL thì lỗi tiêm SQL có thể đã xảy ra.
- Truy cập bài thực hành tại: http://localhost:8000/injection_sql_lab



SQL injection

Tiêu đề:

Lỗ hổng SQL Injection trên trang đăng nhập của hệ thống.

Mô tả lỗ hổng:

Lỗ hổng này là một kiểu tấn công Injection, trong đó kẻ tấn công chèn mã SQL thông qua dữ liệu đầu vào từ phía người dùng và ảnh hưởng đến việc thực thi các câu lệnh SQL được xác định trước trong ứng dụng. SQL Injection cho phép kẻ tấn công đọc được dữ liệu nhạy cảm từ cơ sở dữ liệu, chỉnh sửa dữ liệu (thêm, sửa, xóa), thực thi các lệnh quản trị trên cơ sở dữ liệu (ví dụ: tắt hệ quản trị cơ sở dữ liệu - DBMS), và trong một số trường hợp có thể thực hiện các lệnh trên hệ điều hành.

Các hậu quả chính của lỗ hổng SQL Injection bao gồm:

- Đọc được thông tin nhạy cảm trong cơ sở dữ liệu.
- Chỉnh sửa dữ liệu (chèn, xóa, cập nhật).
- Thực thi các lệnh quản trị trên cơ sở dữ liệu.
- Trong một số trường hợp có thể kiểm soát máy chủ cơ sở dữ liệu.

Tóm tắt:

Lỗ hổng SQL Injection được phát hiện trên trang đăng nhập của bài thực hành tại

http://localhost:8000/injection_sql_lab. Khi người dùng cố gắng đăng nhập bằng tài khoản admin, có thể nhận ra lỗ hổng này bằng cách chèn ký tự ' vào trường username hoặc password. Nếu hệ thống trả về lỗi SQL, điều đó chứng tỏ ứng dụng dễ bị tấn công SQL Injection.

Bằng cách sử dụng payload: admin cho username và anything' OR '1'='1 cho password, kẻ tấn công có thể bỏ qua cơ chế xác thực và đăng nhập thành công mà không cần biết mật khẩu thực sự của tài khoản admin.

Các bước để thực hiện lại và bằng chứng:

Lab 1: Tổng quan các lỗ hổng web thường gặp

1. **Bước 1:** Truy cập vào trang web bài thực hành tại `http://localhost:8000/injection_sql_lab`.
 - Màn hình hiển thị trang đăng nhập với hai trường username và password.
2. **Bước 2:** Trong trường username, nhập admin và trong trường password, nhập: anything' OR '1'='1'.

- **Giải thích:** Câu lệnh SQL được tạo ra sẽ có dạng:

```
SELECT * FROM introduction_login WHERE user='admin' AND password='anything' OR '1'='1';
```

- Đoạn mã OR '1'='1' khiến cho điều kiện password luôn đúng, bất kể giá trị anything là gì.
3. **Bước 3:** Nhấn nút Login và quan sát kết quả.
 - Kết quả: Người dùng có thể đăng nhập thành công mà không cần biết mật khẩu của tài khoản admin.

Tài liệu hỗ trợ và tham khảo:

- OWASP SQL Injection Prevention Cheat Sheet - Các phương pháp phòng chống SQL Injection.
- Common Weakness Enumeration (CWE) - Các lỗ hổng liên quan đến Injection:
 - **CWE-89:** SQL Injection.
 - **CWE-20:** Thiếu kiểm tra và xác thực dữ liệu đầu vào.

Mức độ ảnh hưởng của lỗ hổng:

Lỗ hổng này có mức độ nghiêm trọng **cao** vì SQL Injection có thể cho phép kẻ tấn công:

1. **Bỏ qua cơ chế xác thực** và đăng nhập vào hệ thống mà không cần cung cấp mật khẩu thực tế.
2. **Trích xuất và sửa đổi dữ liệu nhạy cảm** trong cơ sở dữ liệu, bao gồm các thông tin cá nhân, thông tin tài chính hoặc các thông tin bảo mật khác.
3. **Thực thi các lệnh SQL độc hại** như xóa toàn bộ bảng (DROP TABLE) hoặc thêm tài khoản quản trị viên (INSERT).
4. **Chiếm quyền điều khiển máy chủ cơ sở dữ liệu**, nếu kẻ tấn công có thể thực hiện các lệnh quản trị.

Khuyến cáo khắc phục:

1. **Sử dụng câu truy vấn có tham số (Parameterized Queries):**
 - Không truyền trực tiếp chuỗi nhập liệu của người dùng vào câu lệnh SQL. Thay vào đó, sử dụng các câu truy vấn có tham số để ngăn chặn việc chèn mã SQL. Ví dụ:

```
SELECT * FROM users WHERE username = ? AND password = ?;
```
 - Việc sử dụng câu lệnh có tham số đảm bảo rằng dữ liệu đầu vào luôn được xử lý dưới dạng chuỗi thay vì một phần của câu lệnh SQL.
2. **Sử dụng ORM (Object-Relational Mapping):**
 - Sử dụng các thư viện ORM như Hibernate, Entity Framework để quản lý các truy vấn thay vì viết thủ công các câu lệnh SQL. ORM sẽ tự động tránh việc chèn các lệnh SQL độc hại.
3. **Kiểm tra và lọc dữ liệu đầu vào:**
 - Kiểm tra tất cả các trường nhập liệu của người dùng để đảm bảo không có ký tự đặc biệt hoặc mã SQL nào được đưa vào truy vấn. Có thể sử dụng các hàm lọc như `addslashes()` hoặc `mysqli_real_escape_string()` trong PHP hoặc các hàm tương tự trong các ngôn ngữ khác.
4. **Áp dụng cơ chế xác thực nhiều lớp (2FA):**
 - Kết hợp với các cơ chế bảo mật khác như xác thực hai lớp (2FA) để giảm thiểu rủi ro ngay cả khi kẻ tấn công vượt qua được lớp xác thực đầu tiên.
5. **Xác thực và hạn chế quyền truy cập:**
 - Đảm bảo rằng các tài khoản cơ sở dữ liệu chỉ có quyền hạn tối thiểu cần thiết. Các tài khoản người dùng không nên có quyền thực thi các lệnh DROP, DELETE, hoặc ALTER trên cơ sở dữ liệu.

Bài tập 4: Insecure Design

Tiêu đề:

Lỗ hổng Thiết Kế Không An Toàn trên hệ thống quản lý vé xem phim.

Mô tả lỗ hổng:

Thiết kế không an toàn là một loại lỗ hổng rộng đại diện cho nhiều điểm yếu khác nhau, thường do thiếu các kiểm soát bảo mật hoặc việc kiểm soát không hiệu quả trong thiết kế của hệ thống.

Lỗ hổng này xuất hiện khi ứng dụng cho phép người dùng nhận vé xem phim miễn phí (tối đa 5 vé), nhưng không kiểm tra kỹ lưỡng số lượng tài khoản mà một người dùng có thể tạo ra. Mặc dù hệ thống giới hạn mỗi tài khoản chỉ được nhận 5 vé miễn phí, nhưng không có biện pháp ngăn chặn việc người dùng tạo nhiều tài khoản để nhận tất cả các vé. Điều này cho phép kẻ tấn công có thể tạo nhiều tài khoản giả mạo, mỗi tài khoản nhận 5 vé miễn phí, và cuối cùng lấy hết tất cả số vé có sẵn trong hệ thống.

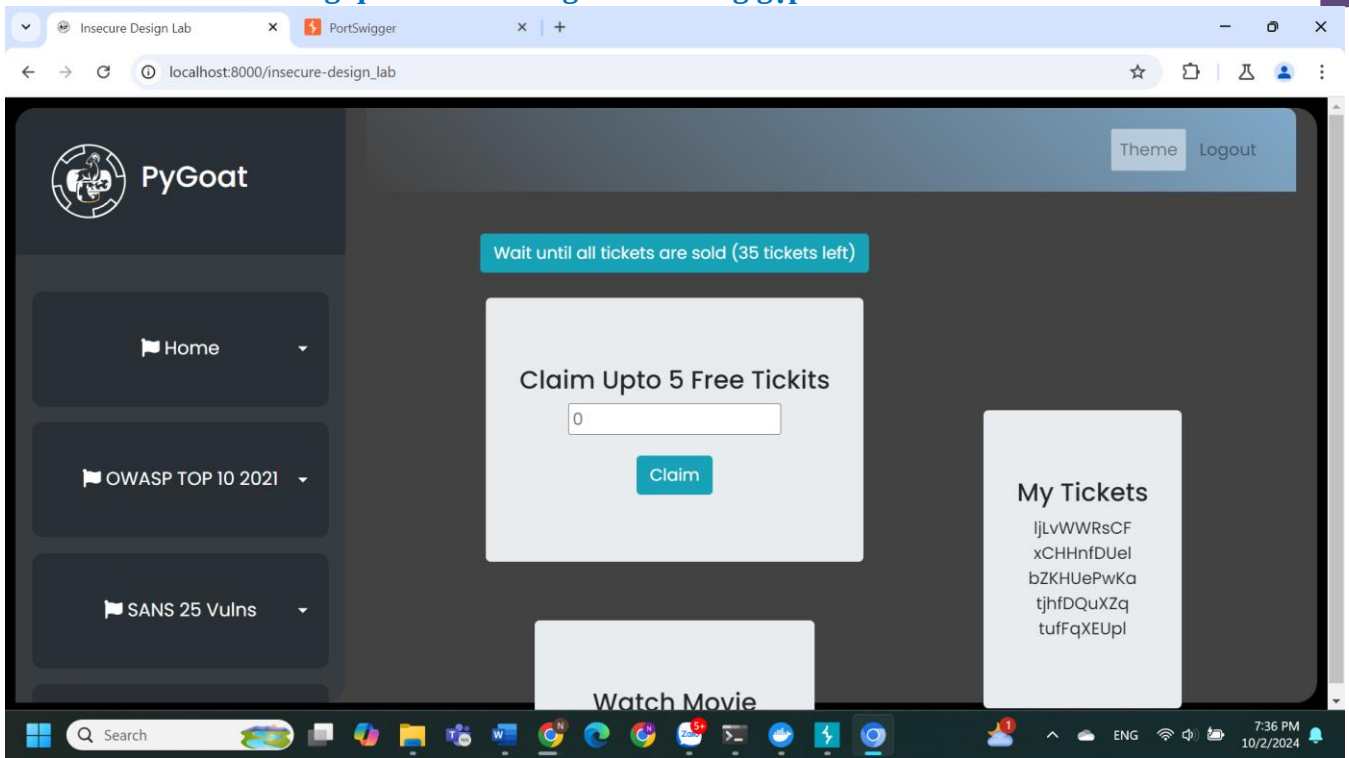
Tóm tắt:

Lỗ hổng được phát hiện trên hệ thống quản lý vé xem phim tại địa chỉ: http://localhost:8000/insecure-design_lab. Ứng dụng yêu cầu người dùng đăng nhập và có một tính năng cho phép người dùng nhận tối đa 5 vé miễn phí sau khi toàn bộ vé được bán hết. Tuy nhiên, lỗi thiết kế nằm ở chỗ không có giới hạn về số lượng tài khoản mà một người dùng có thể tạo. Điều này cho phép kẻ tấn công tạo nhiều tài khoản và mỗi tài khoản đều nhận 5 vé, dẫn đến việc có thể chiếm đoạt toàn bộ vé của hệ thống.

Các bước để thực hiện lại và bằng chứng:

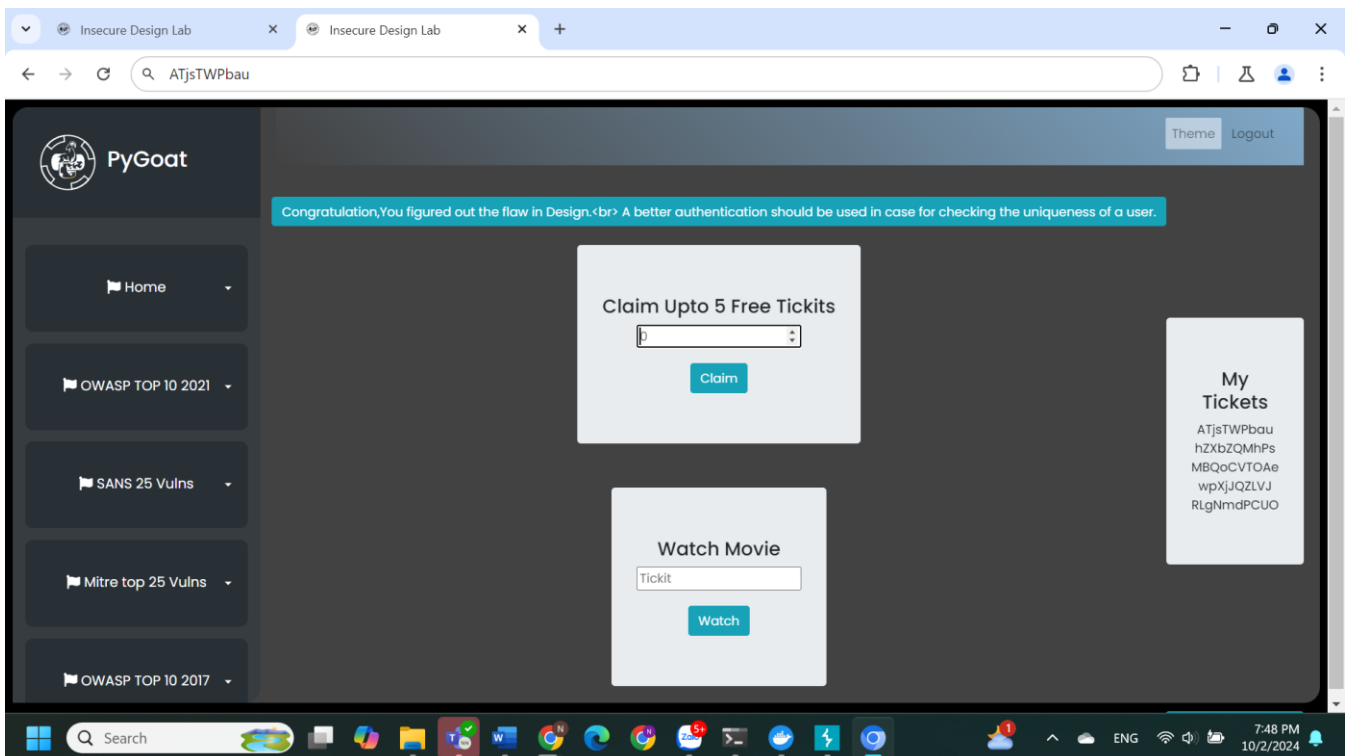
- Bước 1:** Truy cập vào trang web bài thực hành tại: http://localhost:8000/insecure-design_lab.
 - Màn hình hiển thị một form đăng nhập và yêu cầu người dùng nhập tài khoản để bắt đầu.
- Bước 2:** Tạo nhiều tài khoản với thông tin khác nhau (ví dụ: user1, user2, user3,...).
 - Lưu ý:** Mỗi tài khoản có thể đăng ký và nhận tối đa 5 vé miễn phí sau khi toàn bộ vé đã được bán hết.
- Bước 3:** Sử dụng từng tài khoản để nhận vé bằng cách nhấn vào nút "Claim" trên trang quản lý vé.
 - Giải thích:** Hệ thống không có bất kỳ cơ chế kiểm tra nào để giới hạn số lượng tài khoản mà một người có thể tạo, dẫn đến việc có thể tạo vô số tài khoản để chiếm lấy toàn bộ vé.

Lab 1: Tổng quan các lỗ hổng web thường gặp



4. **Bước 4:** Kiểm tra tổng số lượng vé còn lại trên hệ thống sau khi đã sử dụng hết tất cả các tài khoản.

- Nếu mỗi tài khoản nhận được 5 vé, với 12 tài khoản có thể chiếm toàn bộ 60 vé của hệ thống.



Tài liệu hỗ trợ và tham khảo:

- OWASP Insecure Design Cheat Sheet - Các phương pháp ngăn ngừa các lỗi thiết kế không an toàn.
- Common Weakness Enumeration (CWE) - Các lỗ hổng liên quan đến thiết kế không an toàn:

Lab 1: Tổng quan các lỗ hổng web thường gặp

- **CWE-840:** Business Logic Errors.
- **CWE-913:** Improper Control of Dynamically-Managed Code Resources.

Mức độ ảnh hưởng của lỗ hổng:

Lỗ hổng này có mức độ ảnh hưởng **trung bình đến cao** vì lỗi thiết kế không an toàn dẫn đến các vấn đề sau:

1. **Lạm dụng tài nguyên của hệ thống:**

- Kẻ tấn công có thể sử dụng một số lượng lớn tài khoản để chiếm hết tất cả các vé xem phim, khiến các người dùng hợp lệ khác không còn cơ hội nhận vé.

2. **Vì phạm các quy tắc và chính sách của hệ thống:**

- Mặc dù quy tắc là mỗi người chỉ được nhận tối đa 5 vé, nhưng một người có thể lợi dụng việc thiết kế thiếu bảo mật để nhận số lượng vé vượt quá giới hạn.

3. **Mất uy tín và gây thiệt hại tài chính:**

- Do kẻ tấn công có thể sử dụng các vé này để xem phim miễn phí mà không tuân theo các quy định của hệ thống, dẫn đến việc mất doanh thu và gây thiệt hại tài chính cho đơn vị quản lý.

#Khuyến cáo khắc phục:

1. **Giới hạn số lượng tài khoản mà một người dùng có thể tạo:**

- Sử dụng các cơ chế như xác minh danh tính, xác thực bằng email hoặc số điện thoại để giới hạn việc tạo tài khoản ảo. Đảm bảo rằng một người dùng không thể tạo nhiều tài khoản bằng cùng một thông tin danh tính.

2. **Áp dụng cơ chế kiểm tra giới hạn số lượng vé trên toàn hệ thống:**

- Cập nhật logic kinh doanh để kiểm tra tổng số lượng vé mà một người dùng có thể sở hữu trên toàn hệ thống, thay vì chỉ kiểm tra trên một tài khoản đơn lẻ.

3. **Tăng cường các kiểm soát bảo mật ở cấp độ thiết kế:**

- Thực hiện kiểm tra bảo mật trong giai đoạn thiết kế để đảm bảo rằng tất cả các quy định và chính sách bảo mật đã được đưa vào và không bị bỏ qua.

4. **Áp dụng giới hạn thời gian:**

- Áp dụng giới hạn thời gian cho việc nhận vé (ví dụ: mỗi tài khoản chỉ có thể nhận vé trong một khoảng thời gian nhất định sau khi vé được phát hành), hoặc yêu cầu người dùng phải đợi một khoảng thời gian ngẫu nhiên trước khi có thể nhận vé.

5. **Kiểm tra định kỳ các lỗ hổng thiết kế:**

- Thực hiện các bài kiểm tra bảo mật định kỳ và đánh giá thiết kế tổng thể để xác định các điểm yếu có thể xuất hiện trong quá trình phát triển và triển khai.

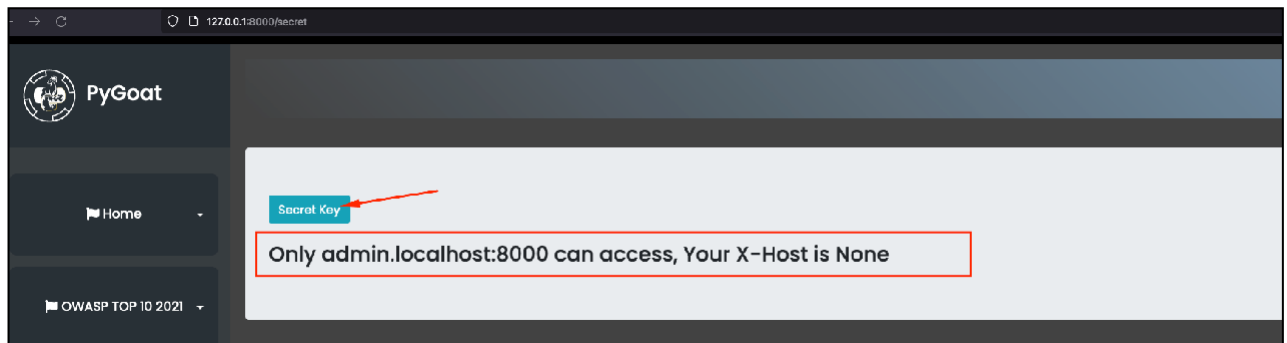
Bài tập 5: Security Misconfiguration

a. Mô tả

- Ứng dụng có thể gặp phải vấn đề cấu hình bảo mật sai trong một số trường hợp như:
- Các tính năng không cần thiết được bật hoặc cài đặt (ví dụ như các cổng, dịch vụ, trang, tài khoản, hoặc quyền không cần thiết)
- Tài khoản mặc định và mật khẩu được bật và không thay đổi.
- Xử lý thông báo lỗi quá mức cho người dùng
- Đối với các hệ thống được nâng cấp, tính năng bảo mật mới nhất bị vô hiệu hoá hoặc không được cấu hình an toàn.
- Phần mềm bị lỗi thời hoặc có lỗ hổng (xem thêm A06:2021-Vulnerable and Outdated Components)

b. Kịch bản tấn công thực hành

- Kịch bản thực hành này sẽ có một chức năng để lấy khoá bí mật, tuy nhiên nó chỉ có thể truy cập bởi người dùng quản trị.
- Truy cập bài tập tại: http://localhost:8000/sec_mis_lab
- Kết quả khi nhấn lấy khoá bí mật, thông báo hiện ra là chỉ có trang `admin.localhost:8000` mới có thể truy cập vào chức năng này. Và thông báo X-Host lúc này là `None`



- Khi trở thành người dùng quản trị, khoá bí mật có thể nhìn thấy.

Tiêu đề: Security Misconfiguration in Key Retrieval Function on Admin-Only Page

Mô tả lỗ hổng:

Lỗ hổng này là một **Security Misconfiguration**. Chức năng lấy khoá bí mật chỉ có thể truy cập bởi người dùng quản trị trên trang admin.localhost:8000. Tuy nhiên, ứng dụng không được cấu hình đúng để xử lý kiểm tra quyền truy cập và bảo mật trang. Điều này có thể dẫn đến việc những kẻ tấn công có thể bỏ qua kiểm tra quyền hạn hoặc phát hiện thông tin về cấu hình hệ thống thông qua lỗi xử lý.

Tóm tắt:

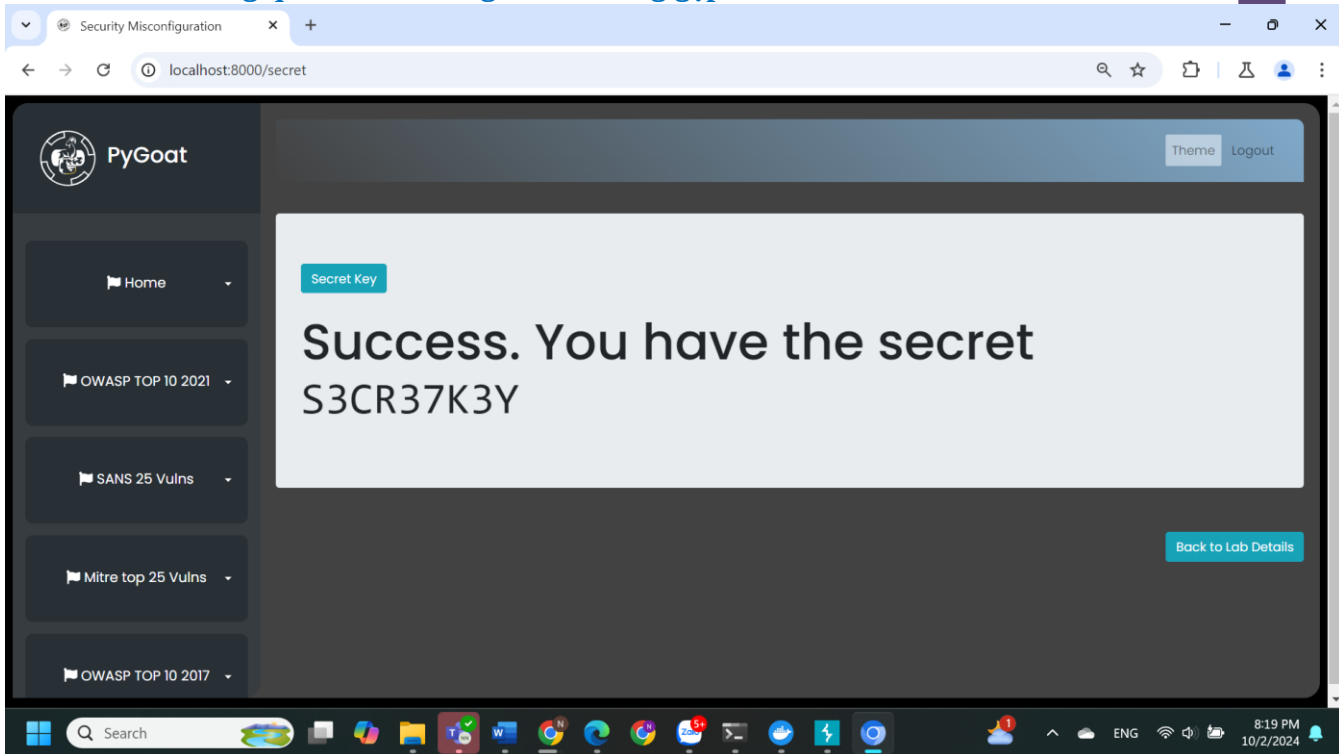
- Ứng dụng gặp phải vấn đề bảo mật khi khoá bí mật có thể được truy cập bởi tài khoản quản trị viên mà không có biện pháp bảo mật đủ mạnh để ngăn chặn tấn công từ bên ngoài.
- Một cảnh báo về X-Host hiển thị None có thể là dấu hiệu của việc không cấu hình đúng lớp bảo mật hoặc không có cơ chế bảo vệ chống lại việc giả mạo yêu cầu.

Các bước để thực hiện lại và bằng chứng:

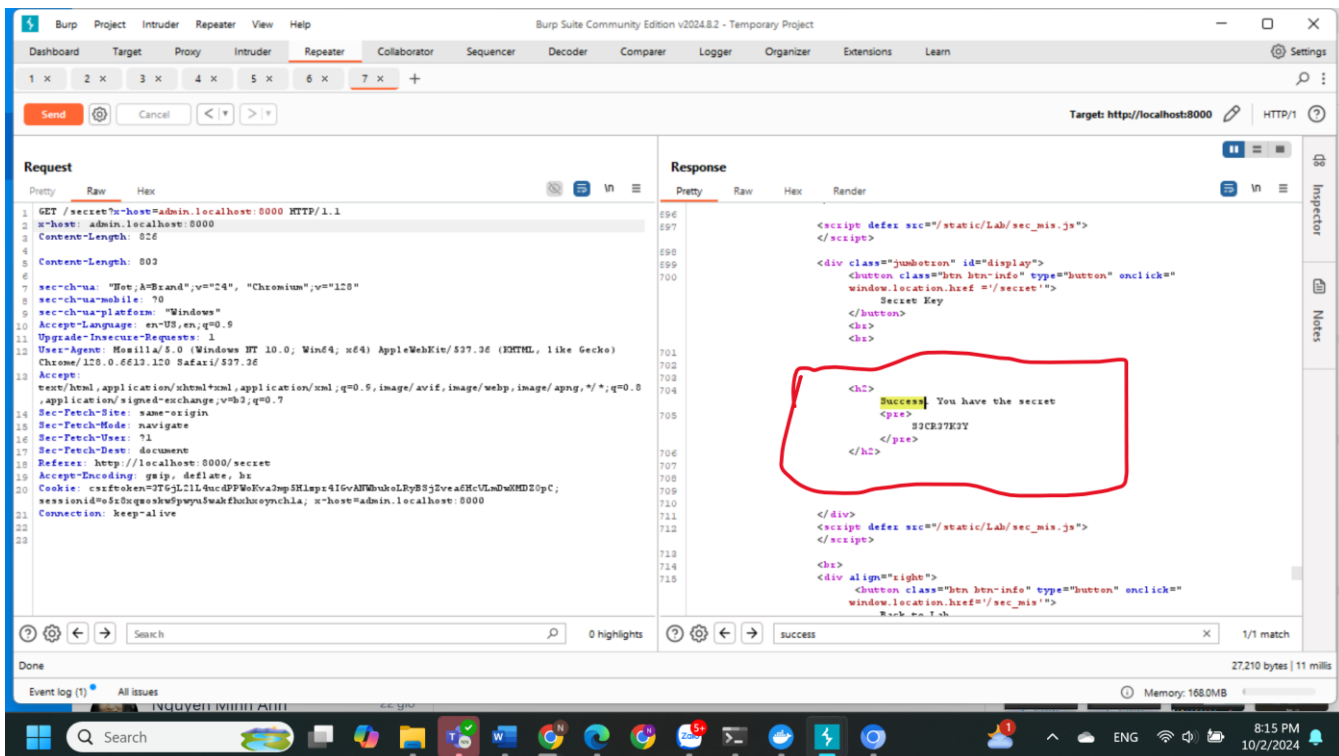
1. Truy cập vào trang chức năng tại địa chỉ: http://localhost:8000/sec_mis_lab.
2. Cố gắng lấy khoá bí mật từ giao diện người dùng không phải quản trị. Thông báo sẽ xuất hiện: "chỉ có trang admin.localhost:8000 mới có thể truy cập vào chức năng này."
3. Quan sát rằng thông tin về X-Host hiển thị None, gợi ý về nơi mình có thể khai thác

Cách 1: cách 1 dùng Burp suite inception on để chặn rồi sửa chữa gói tin

Lab 1: Tổng quan các lỗ hổng web thường gặp



Cách 2: sử dụng repeater



Tài liệu hỗ trợ và tham khảo:

- "A Measurement Study of DNSSEC Misconfigurations"
- "Misconfiguration in Firewalls and Network Access Controls: Literature

Mức độ ảnh hưởng của lỗ hổng:

- Nếu kẻ tấn công có thể lợi dụng cấu hình bảo mật sai này, họ có thể bỏ qua các quyền quản trị để lấy được các khóa bí mật quan trọng.
- Điều này có thể dẫn đến việc xâm phạm các tài sản quan trọng trong hệ thống hoặc mở đường cho các cuộc tấn công leo thang quyền hạn.

Khuyến cáo khắc phục:

- Đảm bảo tất cả các tính năng bảo mật trên các trang quản trị đều được bật và cấu hình đúng.
- Sử dụng phương pháp xác thực hai yếu tố cho các tài khoản quản trị viên.
- Cấu hình đúng các chính sách bảo mật HTTP như kiểm tra X-Host, các quyền hạn truy cập API, và cơ chế bảo vệ chống lại các cuộc tấn công giả mạo.

Bài tập luyện tập

http://localhost:8000/broken_access_lab_2

Tiêu đề: Broken Access Control trong chức năng kiểm soát quyền truy cập của trang Lab 2

Mô tả lỗi hỏng: Lỗi hỏng này là một lỗi hỏng kiểm soát truy cập bị phá vỡ (Broken Access Control) trong trang web thực hành Lab 2. Khi người dùng đăng nhập bằng tài khoản thường và điều chỉnh trường User-agent sao cho nó là user-agent của admin, hệ thống không xác thực quyền hợp lệ mà chấp nhận thay đổi này, dẫn đến việc người dùng thường có thể truy cập vào các trang quản trị mà không có quyền.

Cách làm:

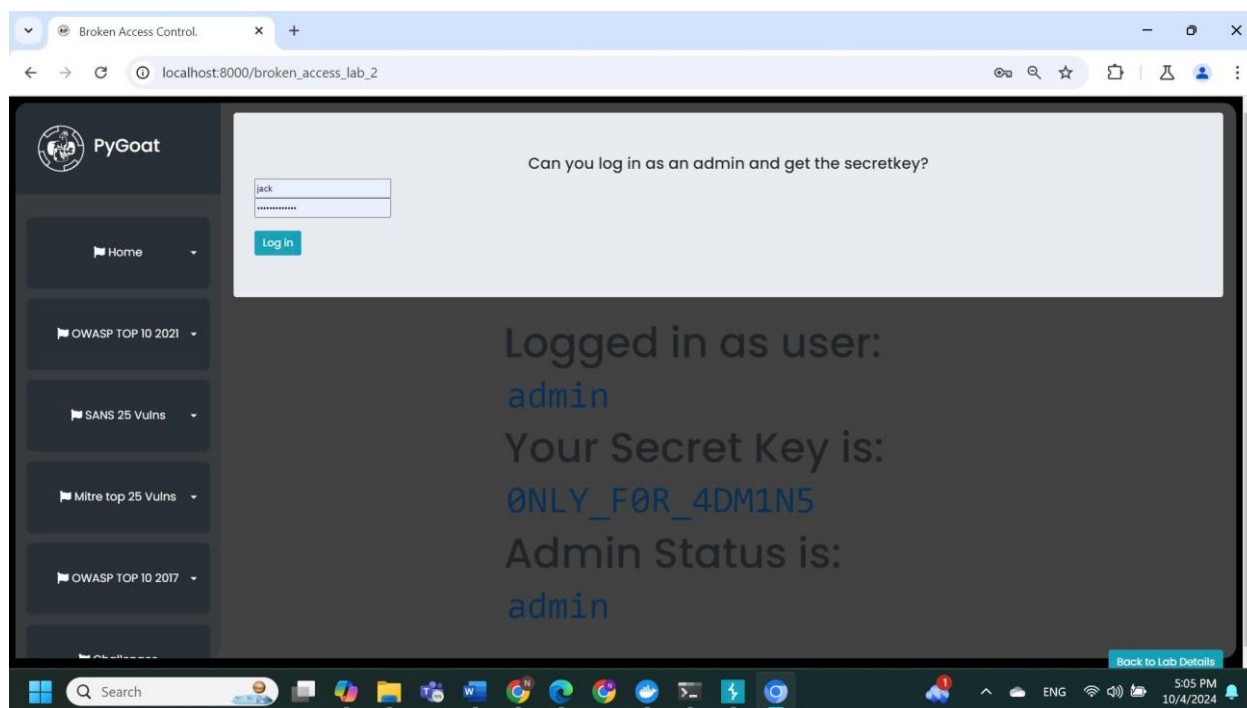
B1: đăng nhập vào, khi này ta chỉ là user bình thường

B2: ta khai thác ở source code, nhấn F12, lúc này ta nhận ra hint

```
</div>
</div>
<br>
<div align="right">
<p></p>
<!--Admins don't use Browsers like Google Chrome or Firefox etc-->
<!--Admins only use pygoat_admin browser-->
</div>
</div>
<!-- jQuery CDN - Slim version (=without AJAX) -->
```

B3: ta bật intercept on và thay đổi giá trị của the user-agent thành pygoat_admin và nhấn gửi

```
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/128.0.6613.120 Safari/537.36
```



Tài liệu hỗ trợ và tham khảo:

- Sử dụng BurpSuite để kiểm tra và chỉnh sửa gói tin.
- Tham khảo tài liệu CORS tại: [PortSwigger - CORS](#)
- Tham khảo nguyên tắc đặc quyền tối thiểu: Principle of Least Privilege

Mức độ ảnh hưởng của lỗ hổng:

Lỗ hổng này cho phép bất kỳ người dùng nào (không cần quyền admin) có thể truy cập và kiểm soát trái phép các tài khoản và thông tin nhạy cảm trên trang web. Từ đó, họ có thể thực hiện các hành vi phá hoại như:

- Truy cập trái phép vào dữ liệu quản trị viên.
- Chỉnh sửa hoặc xóa dữ liệu trên hệ thống.
- Lấy cắp các thông tin nhạy cảm như khóa bí mật.

Khuyến cáo khắc phục:

1. **Ẩn thông tin nhạy cảm khỏi giao diện người dùng:**
 - Không chỉ ẩn liên kết hoặc chức năng trên giao diện mà còn phải đảm bảo quyền hạn được kiểm soát chặt chẽ ở tầng máy chủ.
2. **Xác thực lại quyền hạn người dùng** sau khi thay đổi bất kỳ thuộc tính đặc quyền nào như cookie hoặc user-agent
3. **Không sử dụng các trường nhạy cảm như admin trong source code hay cookie mà không mã hóa**
4. **Áp dụng kiểm tra quyền hạn chặt chẽ** trên tất cả các API và phương thức HTTP như POST, PUT, DELETE.
5. **Sử dụng các kỹ thuật mã hóa hoặc JWT hợp lệ** để bảo vệ giá trị token tránh việc giả mạo.

http://localhost:8000/broken_access_lab_3

Tiêu đề: Broken Access Control trong chức năng kiểm soát quyền truy cập của trang Lab 3

Mô tả lỗ hổng:

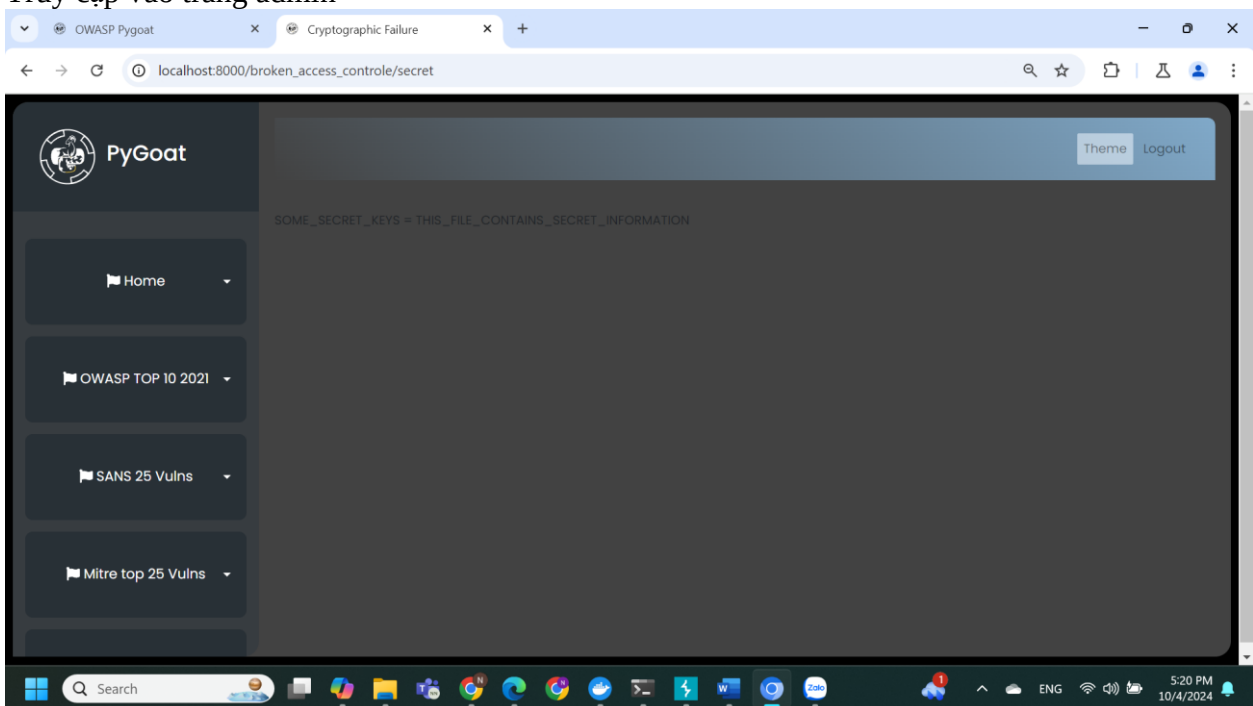
Trang web PyGoat có một lỗ hổng bảo mật thuộc loại Broken Access Control trong Lab 3, cho phép người dùng không có quyền truy cập vào các tài nguyên nhạy cảm mà lẽ ra chỉ dành riêng cho quản trị viên.

Lab 3 Details

```
user : admin
password : admin_pass
this is the admin credential
and
user : John
password : reaper
this is regular user credential
can u get access to admin page contents using regular user credentials or without credentials ?
```

Access Lab 3

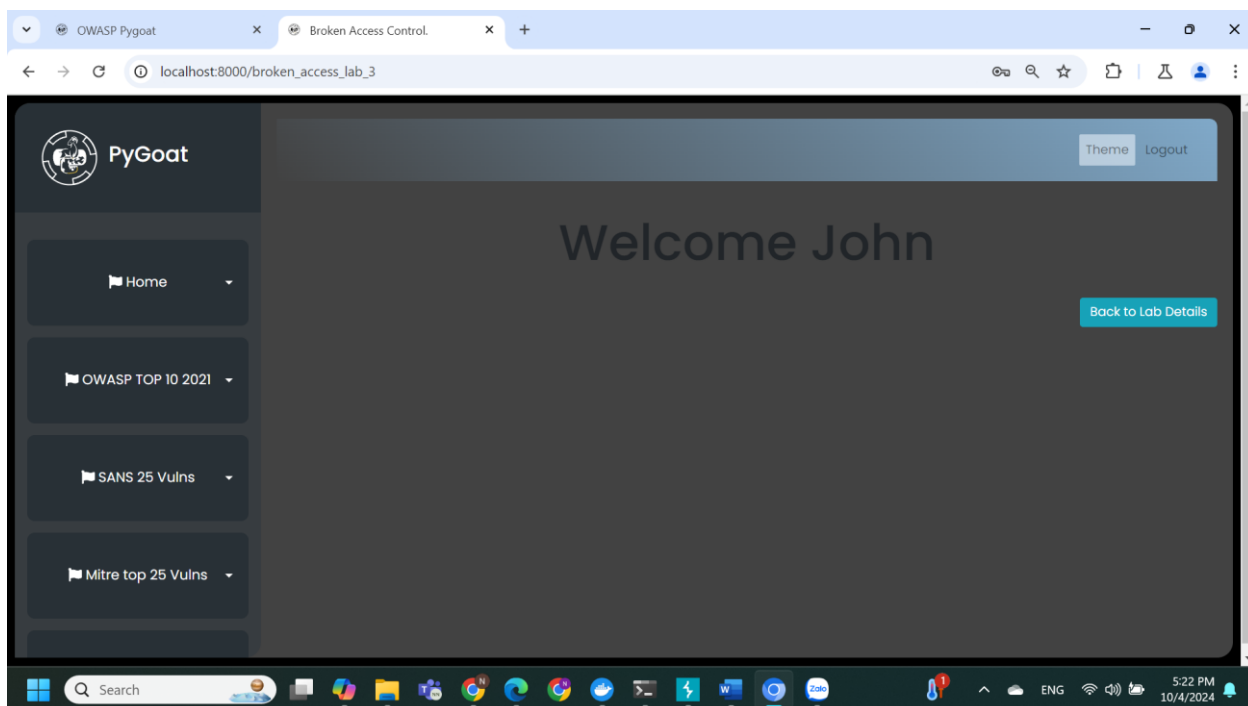
Truy cập vào trang admin



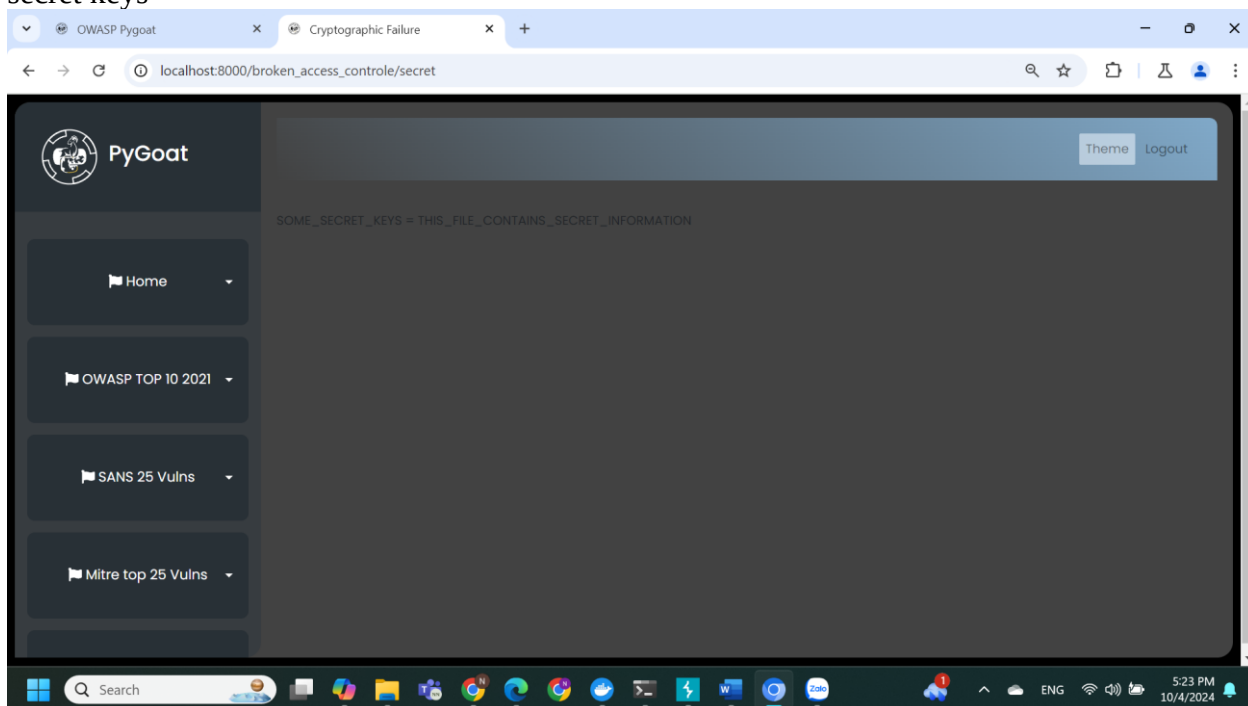
Ta phát hiện ở đây đề 1 thẻ a link tới trang /secret

```
> <nav class="navbar navbar-expand-lg navbar-light bg-light"> ... </nav> (flex)
  <title>Broken Access Control.</title>
  > <div class="container"> (flex)
    <h2>Welcome Admin</h2>
    > <div class="nav2">
      > <div>
        <a href="broken_access_control/secret"> SECRET </a> == $0
      </div>
    </div>
  </div>
  <br>
  > <div align="right"> ... </div>
```

Truy cập vào trang user không nhìn thấy secret



Nhưng nếu ta truy cập vào đường dẫn http://localhost:8000/broken_access_controle/secret ta sẽ thấy secret keys



Nguyên nhân gây ra lỗi hỏng:

- Lỗi này xảy ra do hệ thống không thực hiện kiểm tra quyền hạn truy cập (authorization) trên các đường dẫn cụ thể mà chỉ liên kết đối với người dùng thông thường.
- Việc kiểm tra quyền hạn chỉ được thực hiện ở giao diện người dùng, không được thực hiện chặt chẽ ở phía máy chủ (server-side).

Tài liệu hỗ trợ và tham khảo:

- Burp Suite: Sử dụng để kiểm tra và chỉnh sửa gói tin nhằm tìm kiếm và khai thác các lỗi hỏng bảo mật.
- PortSwigger - CORS: Tài liệu tham khảo về CORS và các kỹ thuật vượt qua kiểm tra quyền hạn.
- Nguyên tắc đặc quyền tối thiểu (Principle of Least Privilege): Đảm bảo rằng người dùng chỉ có

quyền truy cập vào các tài nguyên và chức năng cần thiết, hạn chế tối đa quyền hạn để giảm thiểu rủi ro.

Mức độ ảnh hưởng của lỗ hổng:

Lỗ hổng này cho phép bất kỳ người dùng nào (kể cả người dùng không có quyền quản trị) có thể:

- Truy cập và xem các tài liệu hoặc tài nguyên chỉ dành cho quản trị viên.
- Thực hiện các hành vi trái phép như chỉnh sửa, xóa dữ liệu nhạy cảm.
- Truy cập vào thông tin bảo mật như khóa bí mật, thông tin tài khoản của quản trị viên.
- Từ đó, kẻ tấn công có thể leo thang quyền hạn để chiếm quyền điều khiển hệ thống hoặc phá hoại dữ liệu.

Khuyến cáo khắc phục:

6. Xác thực và kiểm tra quyền hạn ở phía máy chủ:
 - Kiểm tra lại quyền hạn người dùng (user authorization) sau khi thay đổi bất kỳ thuộc tính đặc quyền nào như cookie hoặc token.
7. Không lưu trữ các thông tin nhạy cảm trên cookie:
 - Không sử dụng các trường nhạy cảm như admin=true trong cookie mà không mã hóa hoặc ràng buộc với phiên đăng nhập của người dùng.
8. Áp dụng kiểm tra quyền hạn trên tất cả các API:
 - Áp dụng kiểm tra quyền hạn nghiêm ngặt trên tất cả các API và phương thức HTTP như POST, PUT, DELETE thay vì chỉ dựa vào giao diện.
9. Sử dụng các kỹ thuật mã hóa và xác thực token hợp lệ:
 - Sử dụng mã hóa (encryption) hoặc JWT (JSON Web Token) hợp lệ để bảo vệ giá trị token nhằm tránh việc giả mạo và đánh cắp thông tin đăng nhập.
10. Ẩn thông tin nhạy cảm khỏi giao diện người dùng:
 - Không chỉ ẩn liên kết hoặc chức năng trên giao diện mà còn phải đảm bảo quyền hạn được kiểm soát chặt chẽ ở tầng máy chủ.

http://localhost:8000/cryptographic_failure/lab2

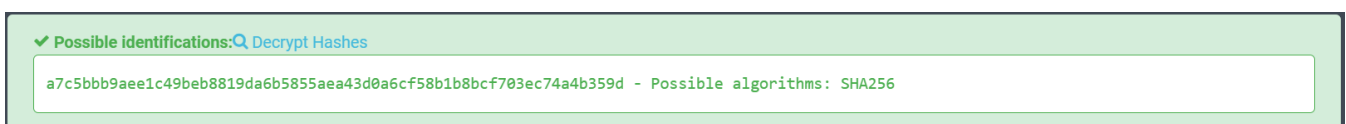
Tiêu đề: Lỗ hổng về việc bảo vệ mật khẩu không an toàn.

Mô tả lỗ hổng: Hacker sử dụng SQL Injection để lấy được bảng CSDL người dùng, trong đó gồm tên User và Password, với Password là 1 chuỗi được băm gồm 64 ký tự (dự đoán có thể là SHA256 hoặc SHA3_256). Tuy nhiên, việc tên tài khoản Admin quá dễ đoán và mật khẩu lại không đủ an toàn đã tạo nên 1 lỗ hổng nguy hiểm vì kẻ tấn công có thể sử dụng các công cụ khác nhau để tìm ra được bản rõ của mật khẩu tài khoản Admin.

Tóm tắt: xác định độ dài ký tự của chuỗi băm mật khẩu tài khoản Admin, từ đó xác định được thuật toán băm. Sau đó, thử nhiều cách khác nhau để tìm được bản rõ của mật khẩu dựa vào nhiều công cụ có sẵn.

Các bước thực hiện và bằng chứng:

Bước 1: Reverse hashed password và truy cập website: hashes.com để biết được mật khẩu dùng hàm hash nào:



Bước 2: Truy cập vào website để [decrypt SHA-256](https://decrypt-sha256.com):

SHA256 Hash

a7c5bbb9aee1c49beb8819da6b5855aea43d0a6cf58b1b8bcf703ec74a4b359d

>>

Text

password777

Elapsed Time: 0.435s

Trial Count: 83K

Bước 3: Kiểm tra lại:

Input

password777

Output

a7c5bbb9aee1c49beb8819da6b5855aea43d0a6cf58b1b8bcf703ec74a4b359d

Kết quả:

 PyGoat

Home

OWASP TOP 10 2021

SANS 25 Vulns

Mitre top 25 Vulns

Theme Login

Successfully logged in as admin

Back to Lab Details

Mức độ ảnh hưởng của lỗ hổng: Lỗ hổng này cho phép kẻ tấn công khôi phục mật khẩu gốc từ dữ liệu đã băm. Một khi truy cập được tài khoản Admin, kẻ tấn công có thể chiếm quyền điều khiển toàn bộ hệ thống, gây ra các vấn đề nghiêm trọng về bảo mật và rò rỉ dữ liệu nhạy cảm.

Khuyến cáo khắc phục:

- + Sử dụng các thuật toán mã hóa mạnh hơn, như Bcrypt, SHA3 để lưu mật khẩu.
- + Cân nhắc thêm salt cho mỗi mật khẩu trước khi băm để tăng độ an toàn.
- + Kiểm tra lại toàn bộ quy trình bảo mật cho dữ liệu nhạy cảm, đặc biệt là mật khẩu.

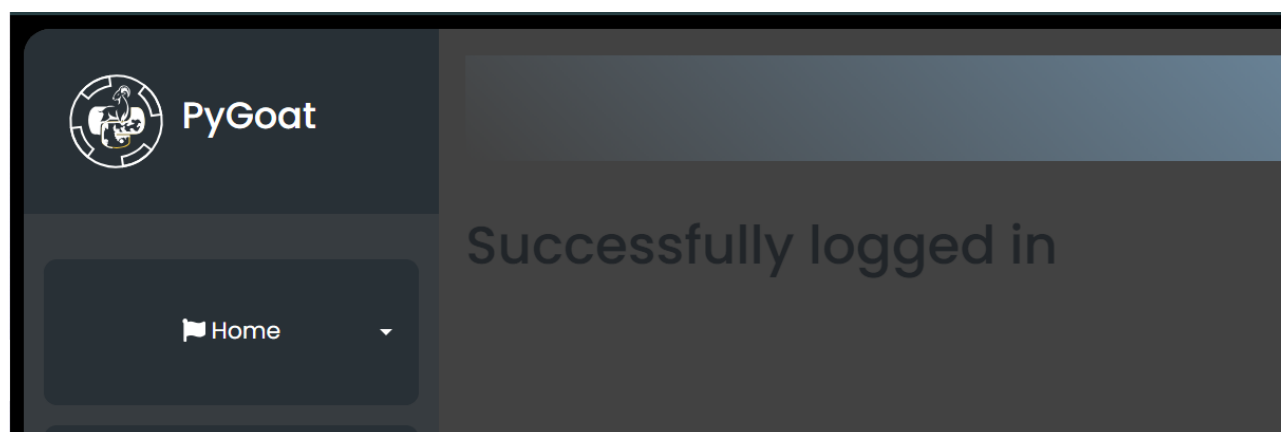
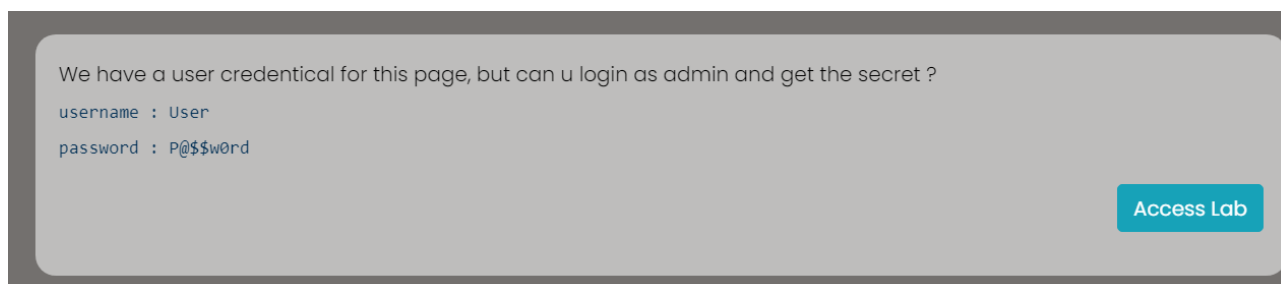
http://localhost:8000/cryptographic_failure/lab3

Mô tả lỗ hổng: Khi thực hiện truy cập vào một đường dẫn bất kỳ nhưng hệ thống không kiểm tra quyền truy cập của tài khoản đó sẽ làm lộ những thông tin bí mật. Người dùng khi muốn tấn công chỉ cần tìm hiểu và biết về đường dẫn đến thông tin cần lấy sẽ có thể dễ dàng lấy được mà không thông qua bất cứ bước xác thực về quyền nào.

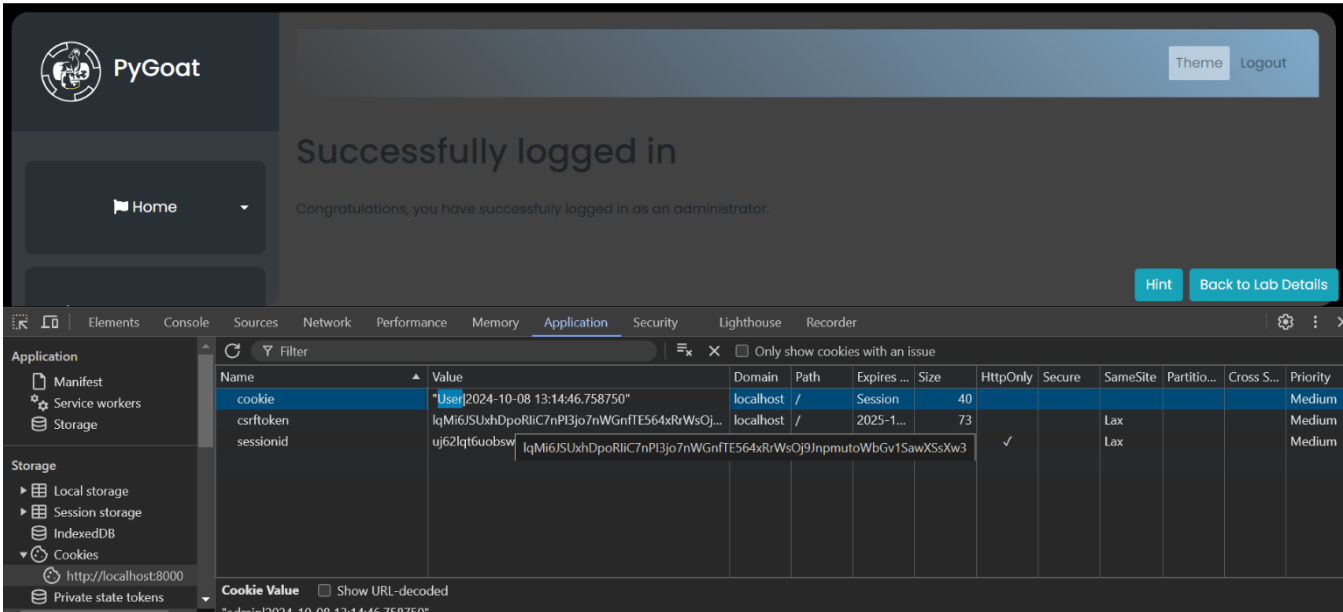
Tóm tắt: Thực hiện tìm đường dẫn đến thông tin bí mật chỉ được truy cập bởi admin, sau đó thực hiện sao lưu đường dẫn đó. Sau đó truy cập vào một tài khoản người dùng bình thường khác và thực hiện truy cập thông qua đường dẫn đã được sao lưu.

Các bước thực hiện:

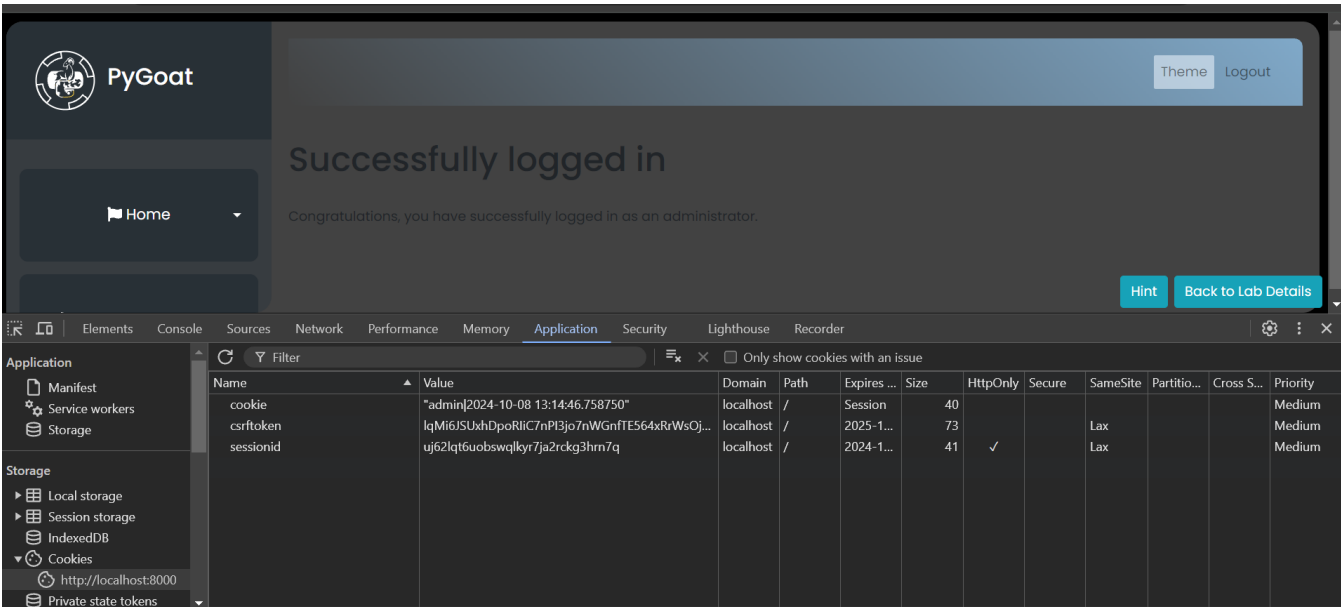
Bước 1: Sau khi đăng nhập vào tài khoản:



Bước 2: Mở Developer Tools -> Application -> Cookies, sửa “User” thành “admin”:



Bước 3: Tải lại trang:

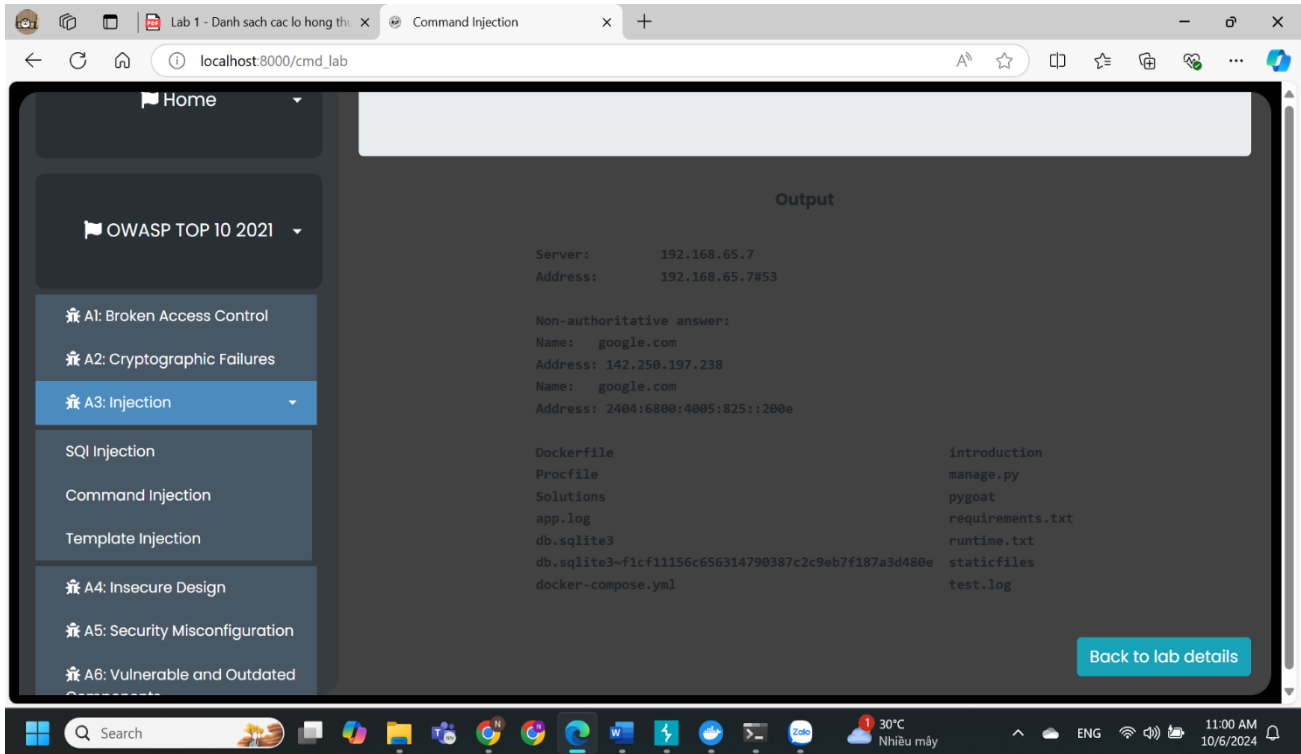


Mức độ ảnh hưởng của lỗ hổng: Việc không kiểm tra quyền truy cập khi thực hiện truy cập bằng đường dẫn đến một trang bất kỳ với mỗi người dùng làm ảnh hưởng đến tính toàn vẹn dữ liệu và bảo mật dữ liệu của hệ thống. Ngoài ra thông qua các đường dẫn, kẻ tấn công cũng có thể khai thác để chiếm quyền hệ thống gây ra những thiệt hại lớn hơn.

Khuyến cáo khắc phục: Sử dụng các biện pháp chứng thực người dùng khi được yêu cầu đưa đến một trang bất kỳ, tại đó phải thực hiện kiểm tra quyền của người dùng trước khi cho phép được truy cập vào các tài nguyên.

http://localhost:8000/cmd_lab

Command injection



Tiêu đề: Command Injection trong tính năng tra cứu Name Server (nslookup) của ứng dụng quản lý tên miền

Mô tả lỗ hổng:

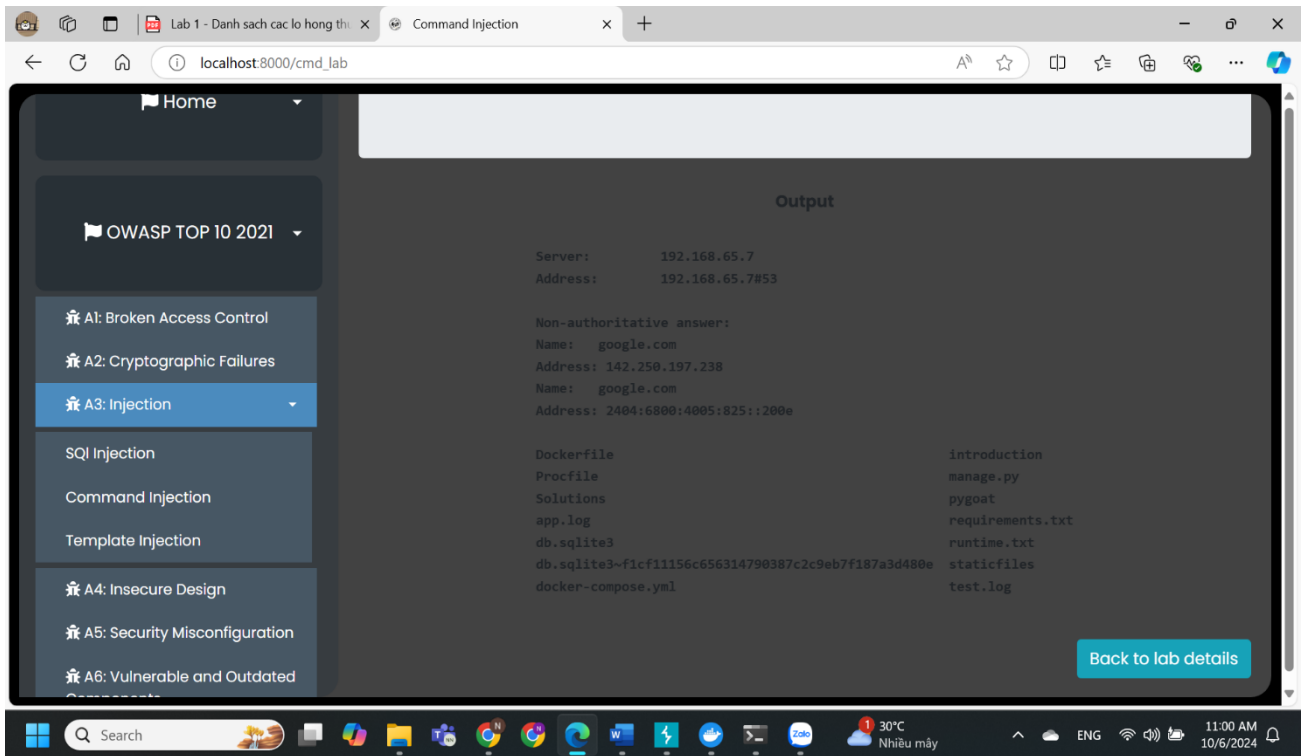
Lỗ hổng **Command Injection** cho phép kẻ tấn công thực thi các lệnh hệ điều hành tùy ý trên máy chủ thông qua tính năng tra cứu tên miền (nslookup) của ứng dụng. Lỗ hổng này xảy ra do dữ liệu đầu vào từ người dùng không được kiểm tra kỹ lưỡng trước khi đưa vào lệnh hệ thống.

Tóm tắt:

Ứng dụng cung cấp tính năng tra cứu thông tin Name Server của một tên miền do người dùng cung cấp. Tuy nhiên, không có kiểm tra hợp lệ đầu vào, nên kẻ tấn công có thể chèn thêm các lệnh tùy ý vào tên miền để thực thi. Ví dụ: google.com && dir trên hệ điều hành Windows sẽ trả về kết quả của cả lệnh nslookup và lệnh dir (liệt kê thư mục).

Các bước để thực hiện lại và bằng chứng:

- Bước 1:** Truy cập vào ứng dụng và tìm đến chức năng tra cứu Name Server (NS Lookup).
- Bước 2:** Nhập tên miền hợp lệ kèm theo chuỗi lệnh bổ sung. Ví dụ: google.com && dir hoặc google.com; ls.
- Bước 3:** Chọn hệ điều hành tương ứng (Windows hoặc Linux) và nhấn vào nút thực thi (Lookup).
- Bước 4:** Quan sát kết quả trả về, bạn sẽ thấy kết quả của cả lệnh nslookup và lệnh bổ sung (dir hoặc ls).



Tài liệu hỗ trợ và tham khảo:

- Ảnh chụp màn hình minh họa kết quả trả về sau khi thực hiện lệnh.
- Mã nguồn của hàm xử lý đầu vào có thể chứa lỗ hổng (ví dụ: sử dụng hàm exec mà không kiểm tra dữ liệu).

Mức độ ảnh hưởng của lỗ hổng:

Lỗ hổng này có mức độ ảnh hưởng nghiêm trọng vì cho phép kẻ tấn công thực thi các lệnh hệ điều hành tùy ý với quyền hạn của ứng dụng, dẫn đến việc truy cập trái phép vào hệ thống, đọc/ghi/xóa file, và có khả năng leo thang đặc quyền nếu các lệnh được khai thác sâu hơn.

Khuyến cáo khắc phục:

- Kiểm tra và lọc đầu vào:** Loại bỏ các ký tự đặc biệt như &&, ;, |, và các chuỗi lệnh nguy hiểm khác.
- Sử dụng API an toàn:** Thay vì sử dụng các hàm thực thi trực tiếp như exec hoặc eval, hãy sử dụng các API an toàn hơn để xử lý các lệnh hệ thống.
- Ngăn chặn đầu vào từ người dùng được đưa trực tiếp vào lệnh hệ thống:** Sử dụng các hàm chuẩn để xử lý lệnh và tránh kết hợp chuỗi mà không kiểm tra.

<http://localhost:8000/ssti/lab>

Tiêu đề: Lỗ hổng Server-Side Template Injection (SSTI) trên Django

Mô tả lỗ hổng:

Lỗ hổng SSTI là một lỗ hổng cho phép kẻ tấn công thực thi mã tùy ý trên máy chủ thông qua việc tiêm mã vào template của ứng dụng Django. Trong trường hợp này, lỗ hổng xảy ra do việc không kiểm tra đầu vào của người dùng, cho phép kẻ tấn công gửi mã template hợp lệ để truy cập thông tin nhạy cảm. Để tái tạo lỗ hổng này, bạn có thể thực hiện các bước sau:

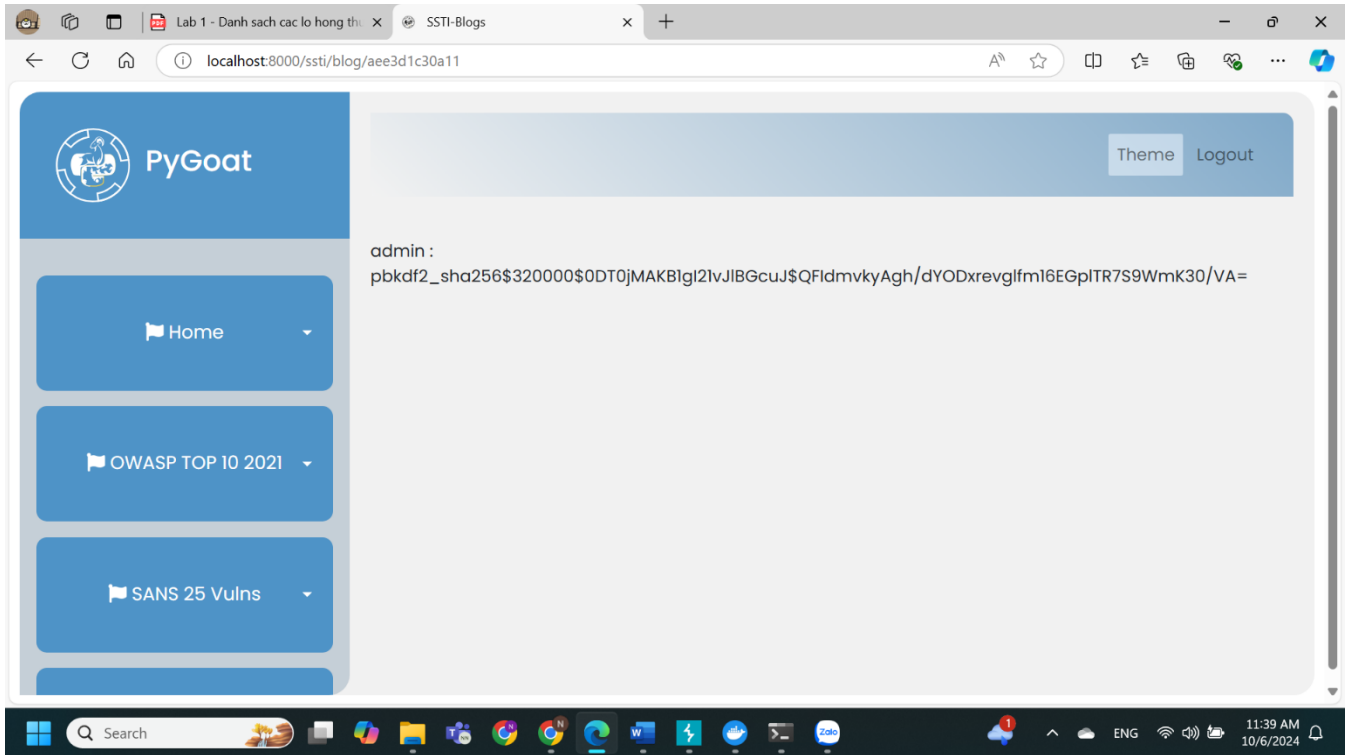
Tóm tắt:

Các bước để thực hiện lại và bằng chứng:

- Đăng nhập vào ứng dụng với quyền truy cập hợp lệ.
- Gửi yêu cầu HTTP GET tới endpoint /ssti/blog/<blog_id> với payload:

```
{% load log %} {% get_admin_log 1 as log %} {% for e in log %} {{ e.user.get_username }} : {{ e.user.password }} {% endfor %}
```

3. Kiểm tra phản hồi từ máy chủ, trong đó sẽ hiển thị tên người dùng và hash mật khẩu của admin.



Tài liệu hỗ trợ và tham khảo:

- Mã nguồn của ứng dụng Django để xác minh lỗ hổng.
- Tài liệu Django về cách hoạt động của template engine và log admin.

Mức độ ảnh hưởng của lỗ hổng:

Lỗ hổng này cho phép kẻ tấn công lấy được thông tin nhạy cảm của người dùng admin, bao gồm hash mật khẩu. Điều này có thể dẫn đến việc kẻ tấn công có thể dễ dàng thực hiện các cuộc tấn công khác như brute-force để chiếm quyền truy cập vào tài khoản admin, ảnh hưởng nghiêm trọng đến bảo mật của ứng dụng.

Khuyến cáo khắc phục:

Để khắc phục lỗ hổng này, bạn cần:

- Kiểm tra đầu vào:** Đảm bảo rằng tất cả dữ liệu đầu vào từ người dùng được kiểm tra và lọc một cách nghiêm ngặt trước khi được đưa vào template.
- Sử dụng thư viện bảo mật:** Sử dụng các thư viện và công cụ bảo mật để phát hiện và ngăn chặn các lỗ hổng SSTI.
- Cập nhật phần mềm:** Luôn cập nhật phiên bản mới nhất của Django và các thư viện liên quan để giảm thiểu các lỗ hổng bảo mật đã biết.

http://localhost:8000/data_exp_lab

Tiêu đề: Lỗ hổng hiển thị thông tin nhạy cảm trên URL /500error của trang web (Sensitive Data Exposure).

Mô tả lỗ hổng:

Lỗ hổng này xảy ra khi truy cập vào một URL không hợp lệ trong ứng dụng, ví dụ /caigido, dẫn đến

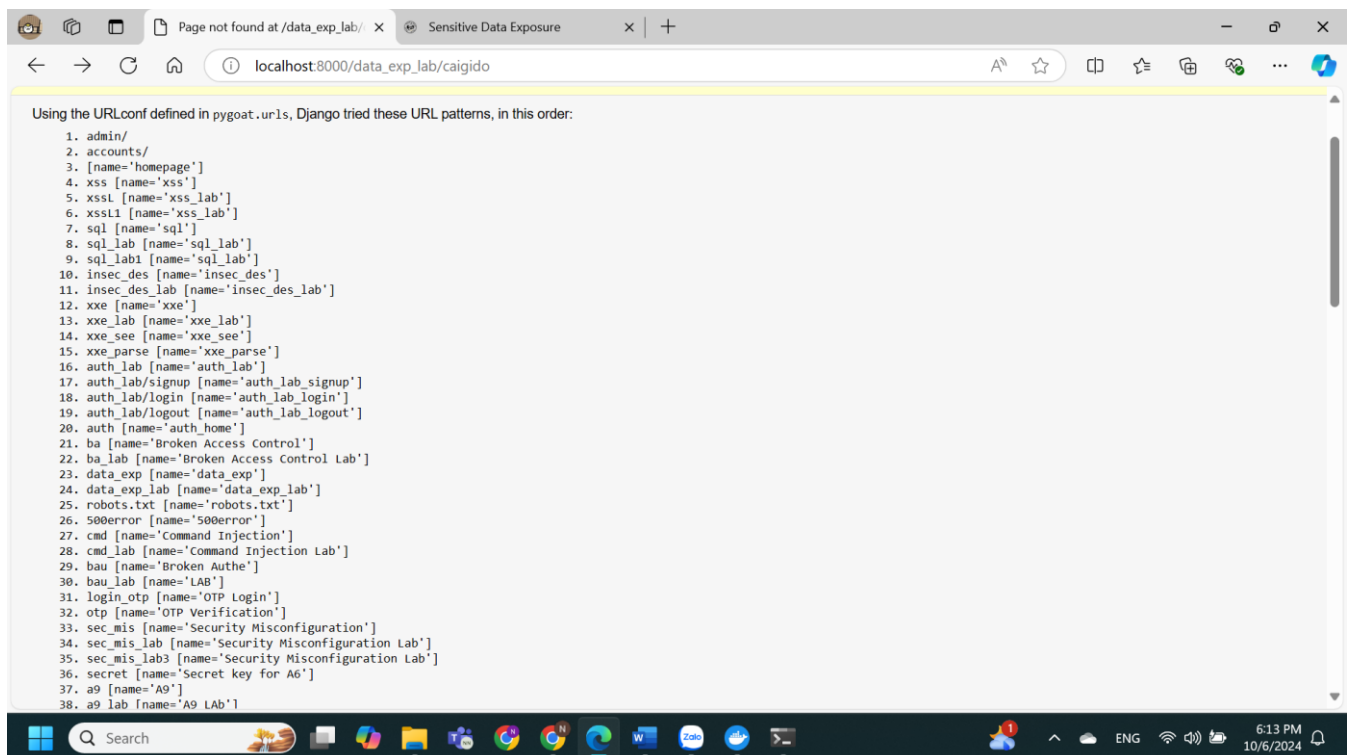
việc trả về danh sách các URL hợp lệ mà ứng dụng hỗ trợ. Trong danh sách này, có một URL là /500error, khi truy cập vào đường dẫn này, ứng dụng trả về trang lỗi 500 và vô tình hiển thị dữ liệu nhạy cảm (SENSITIVE_DATA). Điều này dẫn đến việc lộ thông tin quan trọng có thể bị khai thác bởi kẻ tấn công.

Tóm tắt:

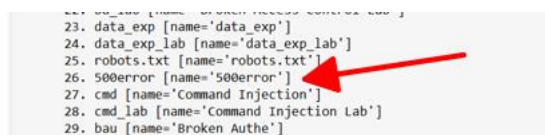
Lỗ hổng này thuộc loại **Sensitive Data Exposure**, cho phép người dùng không xác thực hoặc kẻ tấn công có thể truy cập dữ liệu nhạy cảm của hệ thống thông qua một trang lỗi (Error Page) mà lẽ ra không được phép hiển thị. Điều này xảy ra do việc cấu hình không đúng hoặc không xử lý các thông tin nhạy cảm trước khi trả về kết quả cho người dùng.

Các bước để thực hiện lại và bằng chứng:

- Bước 1:** Truy cập vào bất kỳ URL không hợp lệ nào trên ứng dụng (ví dụ: <http://localhost:8000/caigido>).



- Bước 2:** Ứng dụng sẽ trả về một danh sách các URL hợp lệ mà nó chấp nhận, nhận thấy có một url khả nghi



- Bước 4:** Khi truy cập vào trang này, một lỗi 500 sẽ được hiển thị cùng với thông tin nhạy cảm (SENSITIVE_DATA).

SECURE_REFERRER_POLICY	None
SECURE_SSL_HOST	None
SECURE_SSL_REDIRECT	False
SENSITIVE_DATA	'FLAGTHATNEEDSTOBEFOUND'
SERVER_EMAIL	'root@localhost'

Tài liệu hỗ trợ và tham khảo:

- OWASP: Sensitive Data Exposure
- Tài liệu hướng dẫn xử lý thông tin nhạy cảm của Django (Django Error Handling).

Mức độ ảnh hưởng của lỗ hổng:

- **Cao:** Kẻ tấn công có thể lợi dụng lỗ hổng này để thu thập thông tin nhạy cảm của hệ thống như khóa bí mật, cấu hình cơ sở dữ liệu hoặc các thông tin nội bộ khác, dẫn đến nguy cơ khai thác các lỗ hổng khác.

Khuyến cáo khắc phục:

1. **Ẩn thông tin nhạy cảm:** Đảm bảo không hiển thị dữ liệu nhạy cảm trong các trang lỗi.
2. **Cấu hình chế độ DEBUG = False** trong Django (nếu đang dùng framework này).
3. **Tùy chỉnh trang lỗi 500** để hiển thị thông báo chung chung thay vì chi tiết lỗi.
4. **Xem lại danh sách các URL được công khai** và ẩn các URL chứa thông tin nhạy cảm khỏi danh sách này.

http://localhost:8000/sec_mis_lab3

Mô tả lỗ hổng:

Lỗ hổng này cho phép kẻ tấn công thực hiện việc **JWT Authentication Bypass** thông qua việc **giả mạo token** bằng cách tạo ra JWT với **payload** mang giá trị user: admin. Do ứng dụng không có biện pháp bảo vệ tốt và sử dụng khóa SECRET_COOKIE_KEY để đoán (PYGOAT), kẻ tấn công có thể tạo JWT hợp lệ và truy cập với quyền admin mà không cần xác thực hợp lệ. Điều này có thể dẫn đến việc lộ dữ liệu nhạy cảm, thay đổi thông tin, hoặc chiếm quyền kiểm soát hệ thống.

Tóm tắt:

- Ứng dụng sử dụng JWT để xác thực người dùng nhưng không bảo vệ khóa bí mật (SECRET_COOKIE_KEY) một cách an toàn.
- Khóa bí mật SECRET_COOKIE_KEY là "PYGOAT", một chuỗi dễ đoán, khiến việc giả mạo JWT trở nên dễ dàng.
- Kẻ tấn công có thể tạo JWT mới với payload {"user": "admin"} và thêm nó vào auth_cookie để truy cập với quyền quản trị viên.

Cách làm:

B1: Vì Pygoat là 1 mã nguồn mở nên ta có thể tìm được SECRET_COOKIE_KEY là PYGOAT

B2: JWT là một chuỗi được chia thành 3 phần, phân cách nhau bởi dấu .. Cấu trúc của một JWT như sau:

Header.Payload.Signature

- **Header** (Phần chứa thông tin về thuật toán mã hóa và kiểu dữ liệu)
- **Payload** (Phần chứa thông tin xác thực, dữ liệu của người dùng)
- **Chữ ký (Signature)** của JWT được tính bằng cách kết hợp phần **Header** và **Payload**, sau đó mã hóa bằng SECRET_COOKIE_KEY (ở đây là PYGOAT) và thuật toán HS256.

```
from pygoat.settings import SECRET_COOKIE_KEY

def sec_misconfig_lab3(request):
    if not request.user.is_authenticated:
        return redirect('login')
    try:
        cookie = request.COOKIE["auth_cookie"]
        payload = jwt.decode(cookie, SECRET_COOKIE_KEY, algorithms=['HS256'])
        if payload['user'] == 'admin':
            return render(request, "Lab/sec_mis/sec_mis_lab3.html", {"admin": True})
    except:
        payload = {
            'user': 'not_admin',
            'exp': datetime.datetime.utcnow() + datetime.timedelta(minutes=60),
            'iat': datetime.datetime.utcnow(),
        }

        cookie = jwt.encode(payload, SECRET_COOKIE_KEY, algorithm='HS256')
        response = render(request, "Lab/sec_mis/sec_mis_lab3.html", {"admin": False})
        response.set_cookie(key="auth_cookie", value=cookie)
        return response
```

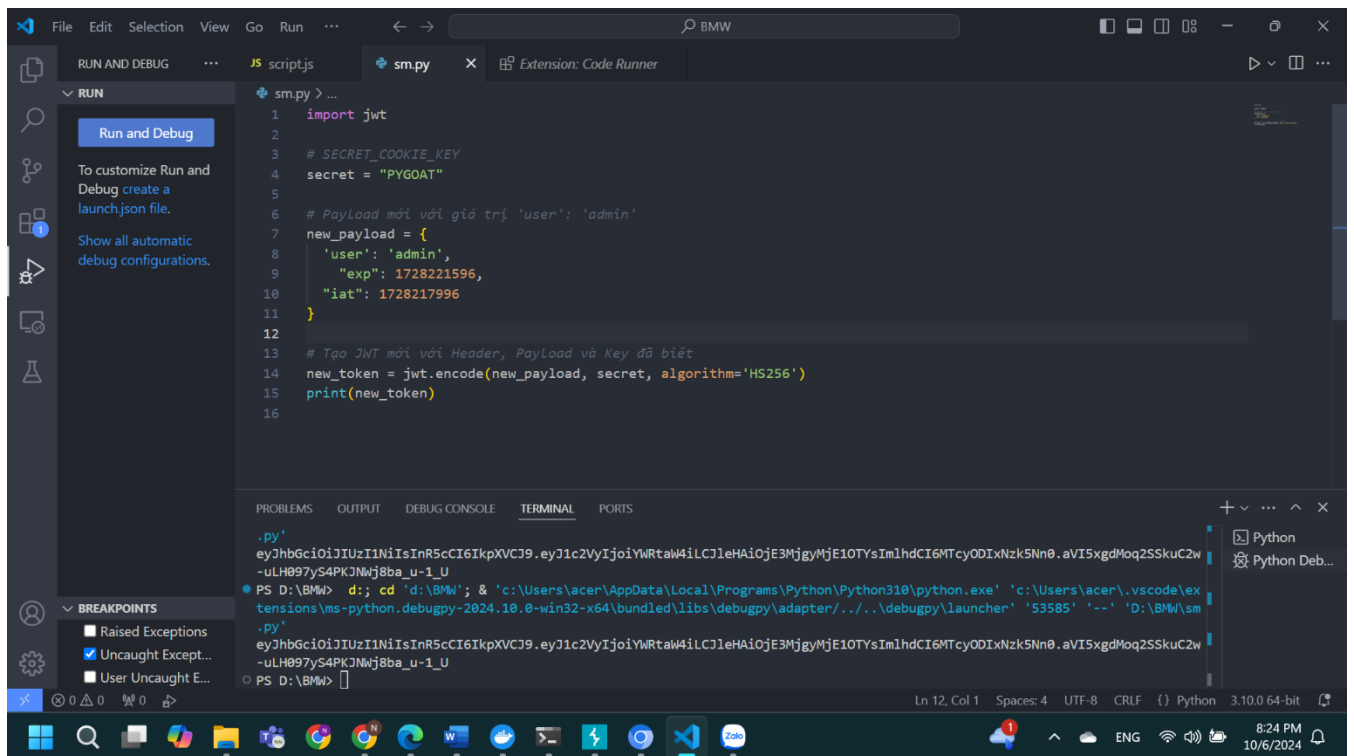
B3: Ta có SECRET_COOKIE_KEY và nhờ burp suit ta có trường cookie được lấy từ auth_cookie

```
Cookie: sessionid=9j5ub7knfykdu8hmivmelnhioqavioe; csrfToken=
GvBAhQTnqLteonT38bYYQs2cpcVIX1m8quqDmst7Pvi7X8Gob6I1Z3I182ff6Hv;
auth_cookie=
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoibm90X2FkbWl1IiwiaXhwIj
oxNzI0MjYyOTY0IiwiaWF0IjE2MjYyOTY0MTc5Lm53x0hy-L5Y1xa6xW1lo52grql_Auif
mACpxvcqJp8
Connection: keep-alive
```

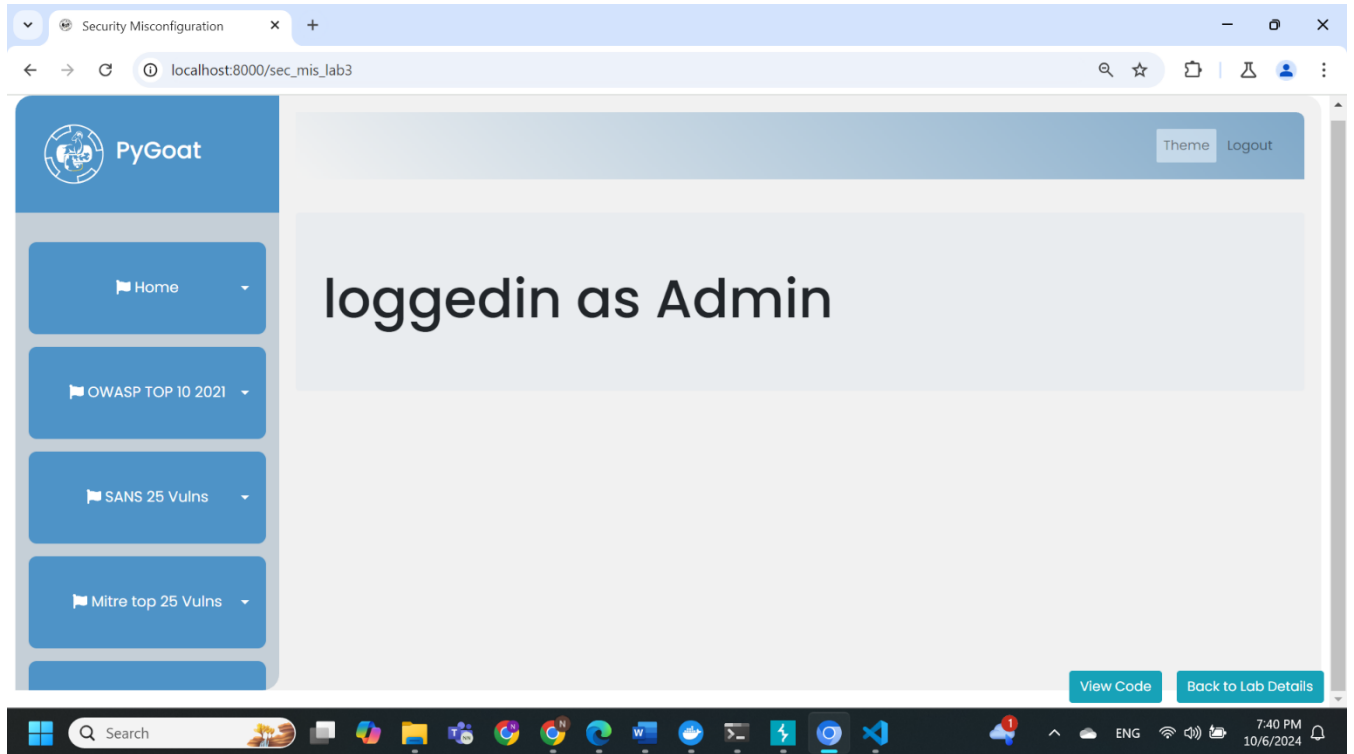
⇒ Mục tiêu: tính được payload sao cho thông tin của nó là admin thay vì là not_admin

B4: Mở burp suite và thay đổi trường auth_cookie bằng cách bật intercept

B5: tính auth_cookie của admin:



B6: thay đổi trường auth_cookie rồi bấm forward gói tin, browser sẽ trả về



Thành công

Tài liệu hỗ trợ và tham khảo:

- [JWT.io Debugger](https://jwt.io) – Để kiểm tra và tạo token JWT.
- Ảnh chụp màn hình chứng minh:

Mức độ ảnh hưởng của lỗ hổng:

Lỗ hổng này rất nghiêm trọng vì nó cho phép kẻ tấn công giả mạo thông tin xác thực của quản trị viên, từ đó có quyền truy cập vào các chức năng quản trị hoặc dữ liệu nhạy cảm trong hệ thống.

Khuyến cáo khắc phục:

- Thay đổi SECRET_COOKIE_KEY thành chuỗi ngẫu nhiên phức tạp và bảo mật nó.
- Triển khai biện pháp xác thực bổ sung như kiểm tra tính hợp lệ của cookie từ phía máy chủ.
- Thêm các chính sách bảo mật như ký và mã hóa toàn bộ token JWT để ngăn chặn việc sửa đổi trái phép.

<https://portswigger.net/web-security/sql-injection/examining-the-database/lab-listing-database-contents-oracle>

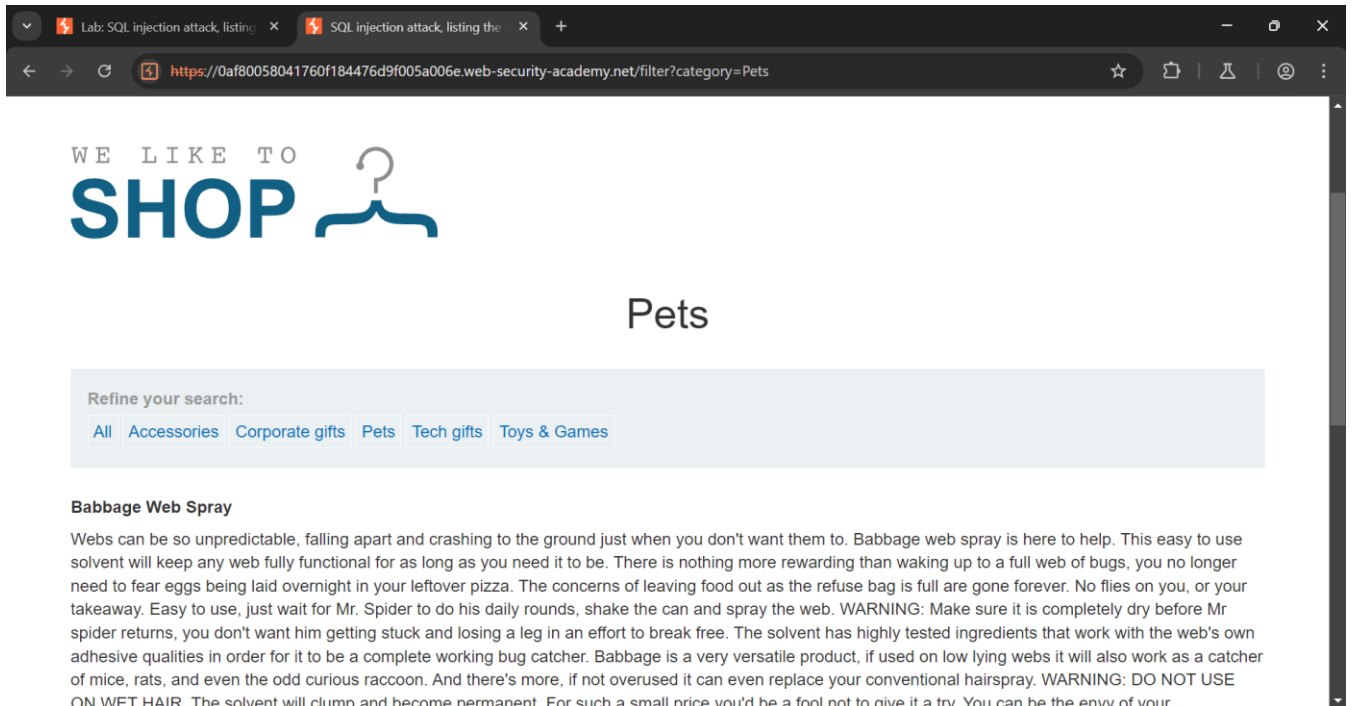
Tiêu đề: Listing Database Contents in Oracle

Mô tả lỗ hổng:

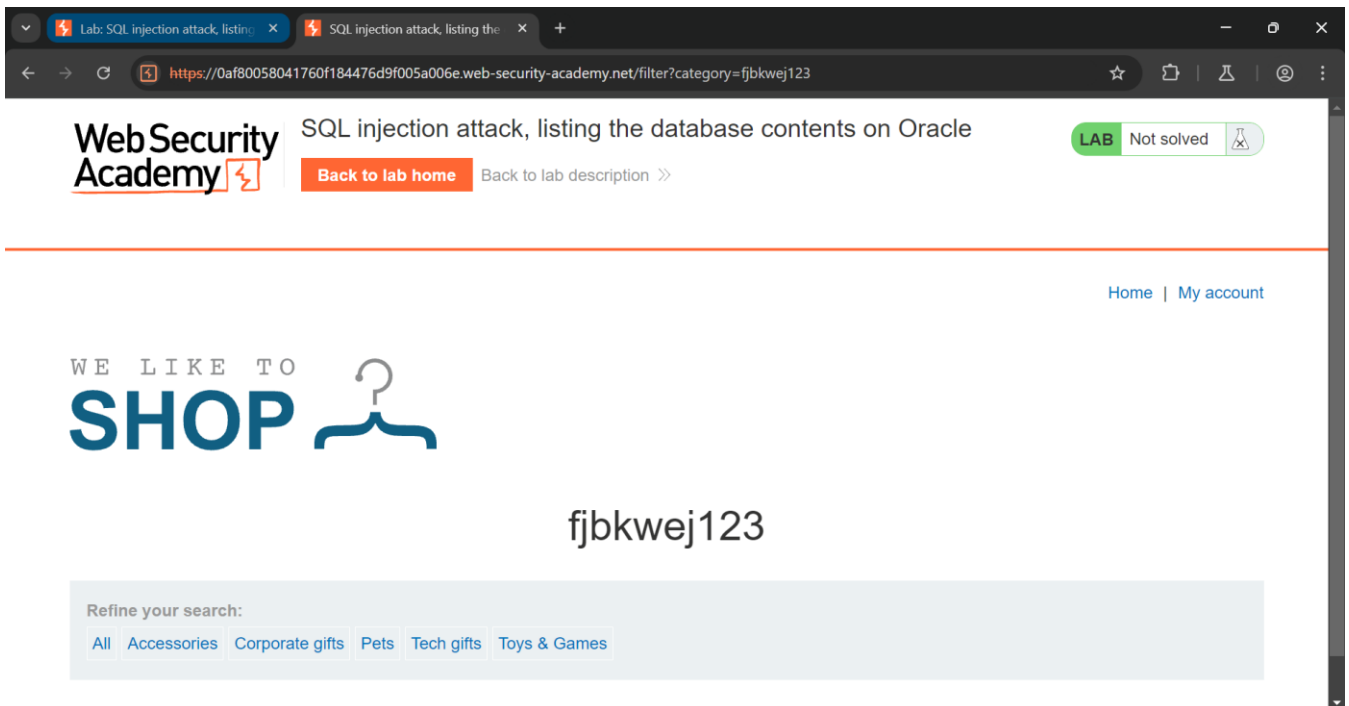
Tóm tắt: Bài lab này chứa lỗ hổng chèn SQL trong bộ lọc danh mục sản phẩm. Kết quả từ truy vấn được trả về trong phản hồi của ứng dụng để ta có thể sử dụng tấn công UNION để lấy dữ liệu từ các bảng khác.

Bước 1: Truy cập vào bài lab, tại phần filter chọn mục bất kì:

Lab 1: Tổng quan các lỗ hổng web thường gặp

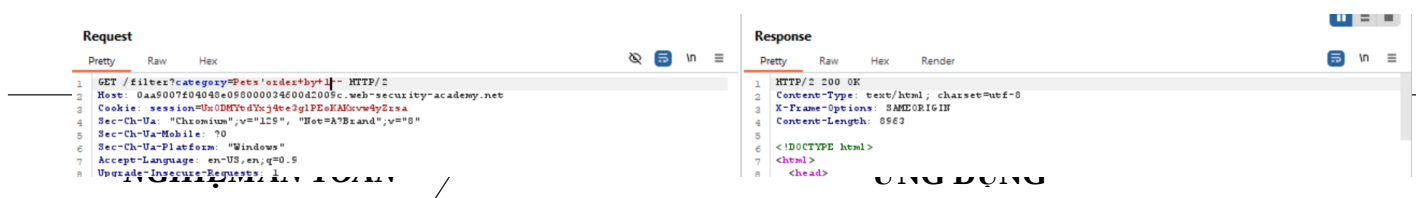


Bước 2: Ta thấy tên danh mục “Pets” xuất hiện trong url. Ta chèn thử tên một danh mục bất kì vào url



- Điều này chứng minh ta có thể xem thông tin của bảng nếu tìm đúng được tên bảng chứa username và password.

Bước 3: Xác định số cột có thể tấn công dễ dàng bằng lệnh ‘order+by+...--

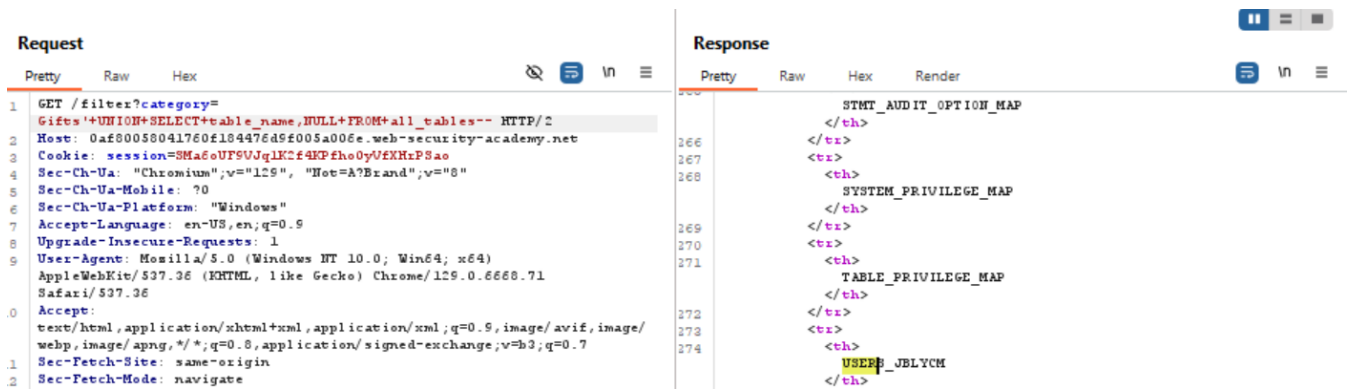




- Order by 3 ta thấy xuất hiện lỗi 500 Internal Server Error => Vậy có 2 cột truy vấn để tấn công.
- Thêm '+UNION+SELECT+'1','1'+FROM+dual-- vào đuôi url để kiểm tra lại:



Bước 4: Thêm lệnh '+UNION+SELECT+table_name,NULL+FROM+all_tables--' vào cuối url để truy xuất tên các bảng:



⇒ Tên bảng cần tìm là: USERS_USMHKI

Bước 5: Truy xuất thông tin của hai cột chứa thông tin username và password bằng lệnh '+UNION+SELECT+column_name,NULL+FROM+all_tables+WHERE+table_name='USERS_USMHKI'--



⇒ Cột chứa username là: USERNAME_BFYVVF

⇒ Cột chứa password là: PASSWORD_LAHVMM

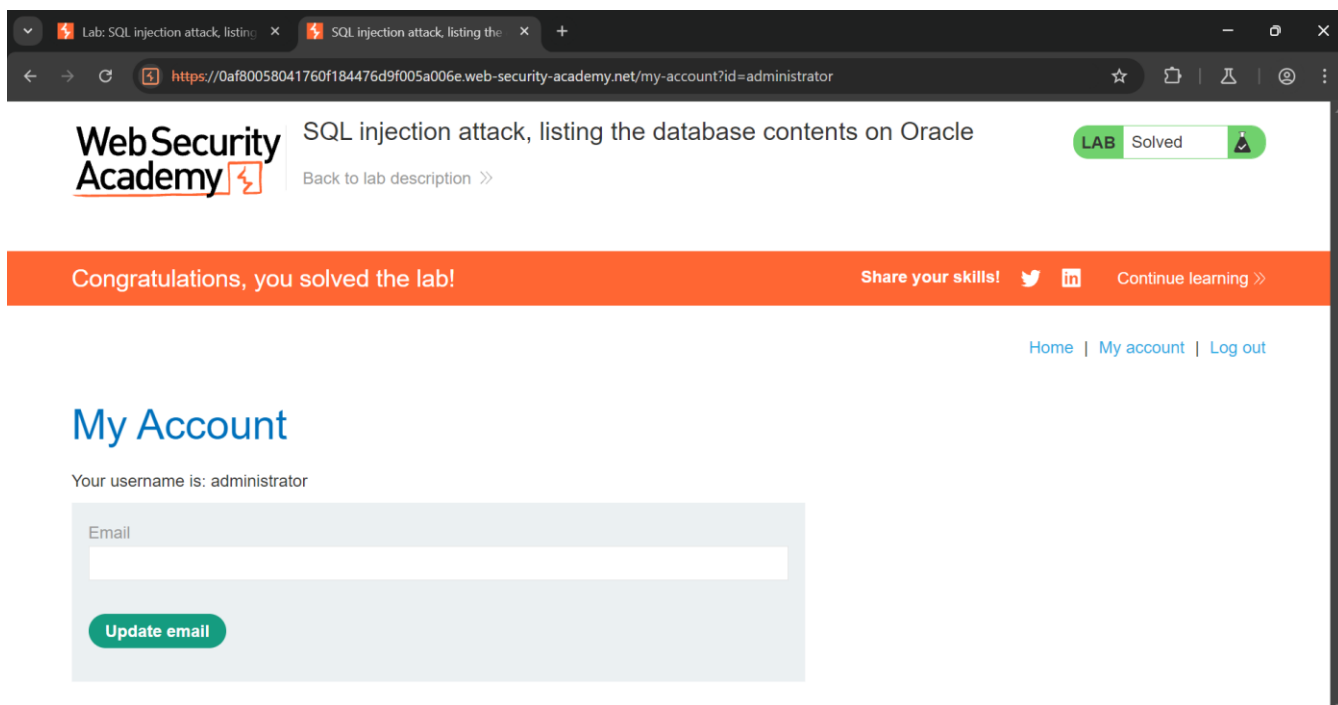
Bước 7: Truy vấn thông tin username và password trong bảng bằng lệnh '+UNION+SELECT+USERNAME_BFYVVF,+PASSWORD_HONEFJ+FROM+USERS_USMHKI--



Username: administrator

Password: gm1chjrvtwiz9sa56y4s

Bước 8: Đăng nhập bằng tài khoản tìm được và nhận kết quả



Mức độ ảnh hưởng của lỗ hổng: Lỗ hổng này cho phép kẻ tấn công kết hợp kết quả từ nhiều bảng cơ sở dữ liệu khác nhau vào truy vấn hiện tại, từ đó tiết lộ thông tin nhạy cảm mà ban đầu không được phép truy cập. Điều này có thể dẫn đến rò rỉ dữ liệu như thông tin người dùng, tài khoản quản trị hoặc cấu trúc của cơ sở dữ liệu, gây thiệt hại tài chính, mất uy tín và tiềm ẩn các rủi ro pháp lý. Ngoài ra, lỗ hổng còn có thể tạo điều kiện cho các cuộc tấn công khác như leo thang đặc quyền hoặc thực thi lệnh trên hệ thống cơ sở dữ liệu.

Khuyến cáo khắc phục:

- + Dùng Prepared Statements hoặc Parameterized Queries: sử dụng câu truy vấn có tham số để bảo vệ dữ liệu đầu vào khỏi việc bị nhúng mã SQL độc hại.
- + Dùng ORM: giúp tránh việc viết truy vấn SQL trực tiếp trong mã.
- + Kiểm tra dữ liệu đầu vào và đầu ra: kiểm tra và xác thực các dữ liệu đầu vào từ người dùng để đảm bảo rằng chúng không chứa các ký tự đặc biệt hoặc mã SQL độc hại.
- + Theo dõi và ghi log hoạt động của hệ thống: Ghi log các hoạt động của hệ thống để phát hiện sớm các hành vi bất thường hoặc các cuộc tấn công SQL Injection

<https://portswigger.net/web-security/file-path-traversal/lab-absolute-path-bypass>

Mô tả: yêu cầu khai thác một lỗ hổng bảo mật để đọc tệp nhạy cảm từ máy chủ, cụ thể là tệp /etc/passwd. Ứng dụng có cơ chế ngăn chặn chuỗi path traversal như ../ (lùi thư mục), nhưng lại cho phép sử dụng đường dẫn tuyệt đối. Điều này cho phép bạn bỏ qua cơ chế bảo vệ và truy cập trực tiếp vào tệp mục tiêu

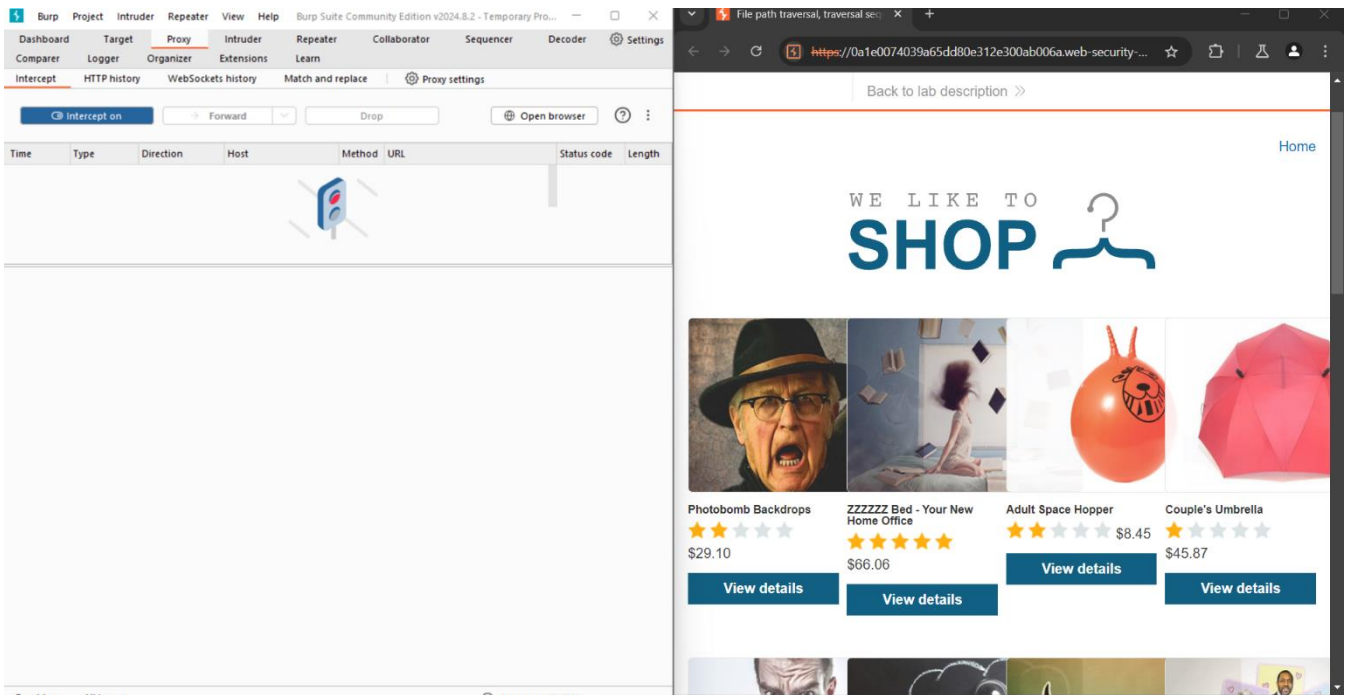
Tóm tắt:

1. **Chặn yêu cầu:** Sử dụng công cụ như Burp Suite để chặn yêu cầu HTTP khi truy vấn ảnh sản

- phản từ trang web.
2. **Thay đổi giá trị tham số:** Thay tham số filename trong yêu cầu thành đường dẫn tuyệt đối như /etc/passwd.
 3. **Gửi lại yêu cầu:** Sau khi sửa, gửi lại yêu cầu và bạn sẽ nhận được nội dung của tệp /etc/passwd trong phản hồi.

Các bước:

B1: Vào burpsuite



B2: Chèn Get /image?filename=/etc/passwd HTTP/

WebSecurity Academy

File path traversal, traversal sequences blocked with absolute path bypass

LAB Not solved

Back to lab description >>

Photobomb Backdrops

★★★★☆

\$29.10

Description:

These backdrops are brought to you in collaboration with our sister company InstaCam. InstaCam is the full package for your social media fake lives. Now you can purchase these add-ons to boost your profile between InstaCam photoshoots.

No technical experience is necessary. The backdrops form part of a software photo suite that can paste you in any pose onto the selected backdrop. Spice up your life with thousands of hilarious photobombing scenarios.

If you would prefer to take your own photos against a painted canvas, and not bother yourself with digital wizardry we can supply you with everything you need. You must be advised the canvases are pretty big and will take up an extraordinary amount of space in your home. Be selective. There is a big second-hand market for these frames, so you can always sell them on, or add to your existing decor with a brand new, exotic, art collection.

We have had incredible feedback for this product, and have been reliably informed that the frames are fully recyclable and have been used in a wide range of DIY projects. Be the envy of your friends and family as they witness your exploits running with photobombing wild horses or apparently dancing with a giraffe (this one is a particularly large canvas). What are you waiting for? Be the person you really want to be today!

B3:

Request

Pretty Raw Hex

```

1 GET /image?filename=/etc/passwd HTTP/2
2 Host:
0a1e0074039a65dd80e312e300ab006a.web-security-academy.
net
3 Cookie: session=V3YUDM6zQz7XroQKwUI8oe42WI75Vx0R
4 Sec-Ch-Ua: "Not;A=Brand";v="24", "Chromium";v="128"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua-Mobile: ?0
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/128.0.6613.120 Safari/537.36
8 Sec-Ch-Ua-Platform: "Windows"
9 Accept:
image/avif,image/webp,image/apng,image/svg+xml,image/*
+/*;q=0.8

```

<https://portswigger.net/web-security/access-control/lab-multi-step-process-with-no-access-control-on-one-step>

- Tiêu đề: Multi-step process with no access control on one step

- Mô tả lỗ hổng:

+ Tóm tắt: Lỗ hổng này xuất hiện khi một quy trình nhiều bước không áp dụng kiểm soát truy cập đầy đủ trên một hoặc nhiều bước trong quy trình, cho phép attacker truy cập trái phép vào các bước bị thiếu kiểm soát mà không cần hoàn thành các bước trước đó.

+ Các bước để thực hiện lại và bằng chứng: dựa theo mục solution của bài lab và mình chứng thực hiện tại video

- Mức độ ảnh hưởng của lỗ hổng: Lỗ hổng này có mức độ nghiêm trọng cao, vì trong một số trường hợp, kẻ tấn công có thể dễ dàng vượt qua các bước xác thực hoặc thanh toán, cho phép họ truy cập trực tiếp vào các quy trình thực hiện hành động trái phép. Hậu quả có thể dẫn đến việc chiếm đoạt tài khoản, thực hiện giao dịch bất hợp pháp, hoặc khai thác tài nguyên nhạy cảm của hệ thống.

- Khuyến cáo khắc phục:

+ Thiết lập kiểm soát truy cập cho từng giai đoạn: Đảm bảo rằng mỗi bước trong quy trình đều được bảo vệ bằng các biện pháp kiểm soát truy cập phù hợp, bao gồm việc xác thực người dùng và kiểm tra trạng thái phiên.

+ Theo dõi trạng thái quy trình: Đảm bảo rằng người dùng phải hoàn thành các bước theo đúng thứ tự, đồng thời lưu trữ và xác nhận trạng thái của quy trình để ngăn chặn việc bỏ qua bất kỳ giai đoạn nào

Bước 1: Đăng nhập tài khoản administrator:admin

The screenshot shows a web browser window with the URL <https://0a3500ea049fb165838fb0bf0...>. The page title is "Multi-step process with no access control on one step" and it says "LAB Solved". Below the title, there is a "Login" form with a "Username" field containing "administrator" and a "Password" field with masked characters. A "Log in" button is at the bottom of the form. To the right of the browser window, the Burp Suite interface is open, showing the "HTTP history" tab. The selected request is a POST to `/admin-roles` with the following body:

```
POST /admin-roles HTTP/2
Host: 0a3500ea049fb165838fb0bf0ca00ca.web-security-academy.net
Cookie: session=6017L5Gxv1fawv0Uwum1Sc5aLb0u3Kvc
Content-Length: 45
Cache-Control: max-age=0
Sec-Ch-Ua: "Chromium";v="125", "Not-A.Brand";v="99"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Origin: https://0a3500ea049fb165838fb0bf0ca00ca.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
```

The response is a 200 OK status with the following headers:

```
HTTP/2 200 Found
Location: /admin
X-Frame-Options: SAMEORIGIN
Content-Length: 0
```


Lab 1: Tổng quan các lỗ hổng web thường gặp

Bước 2: Upgrade quyền của carlos và bắt 2 gói tin khi ấn nút “Upgrade User”, nút “Yes” và thêm chúng vào Repeater

The screenshot shows the Web Security Academy interface on the left and the Burp Suite interface on the right. The Web Security Academy page displays a 'Multi-step process with no access control on one step' lab. The user 'carlos (NORMAL)' is selected, and the 'Upgrade user' button is visible. The Burp Suite interface shows the HTTP history and the details of a POST request to /admin-roles. The request body is a JSON object: {"role": "admin", "password": "12345678", "username": "carlos"}. The response is a 200 OK status with a Content-Type of application/json.

The screenshot shows the Web Security Academy interface on the left and the Burp Suite interface on the right. The Web Security Academy page displays a 'Multi-step process with no access control on one step' lab. The user 'carlos (NORMAL)' is selected, and the 'Upgrade user' button is visible. The Burp Suite interface shows the HTTP history and the details of a POST request to /admin-roles. The request body is a JSON object: {"role": "admin", "password": "12345678", "username": "carlos"}. The response is a 200 OK status with a Content-Type of application/json.

Bước 3: Vào đăng nhập vào tài khoản wiener:peter

The screenshot displays the WebSecurity Academy lab interface on the left and the Burp Suite HTTP history and response on the right.

WebSecurity Academy Lab Interface:

- Lab title: Multi-step process with no access control on one step
- Status: LAB Solved
- Message: Congratulations, you solved the lab!
- Buttons: Share your skills!, Continue learning >>
- Login form:
 - Username: wiener
 - Password: [masked]
 - Log In button

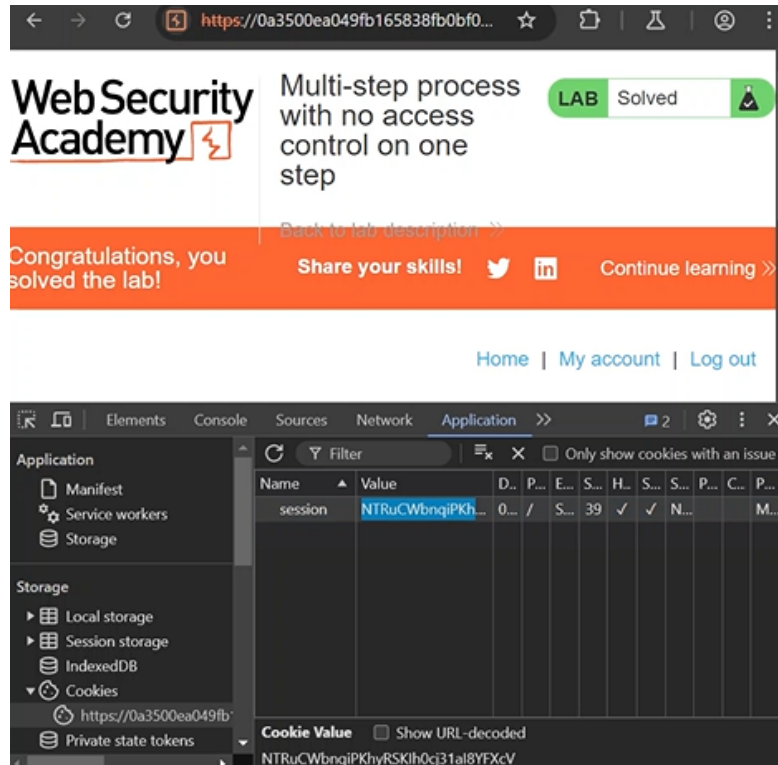
Burp Suite HTTP History:

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type
590	https://0a3500ea049fb165838fb0bf0...	GET	/academyLabHeader			101	147	
589	https://0a3500ea049fb165838fb0...	GET	/login			200	6314	HTML
588	https://0a3500ea049fb165838fb0...	GET	/my-account			302	86	
587	https://0a3500ea049fb165838fb0...	GET	/academyLabHeader			101	147	
581	https://0a3500ea049fb165838fb0...	GET	/resources/images/shop.svg			200	7258	XML
559	https://0a3500ea049fb165838fb0...	GET	/			200	13876	HTML
558	https://0a3500ea049fb165838fb0...	GET	/logout			302	168	
557	https://0a3500ea049fb165838fb0...	GET	/academyLabHeader			101	147	
556	https://0a3500ea049fb165838fb0...	GET	/my-accountId=administrator		✓	200	6402	HTML
555	https://0a3500ea049fb165838fb0...	GET	/academyLabHeader			101	147	

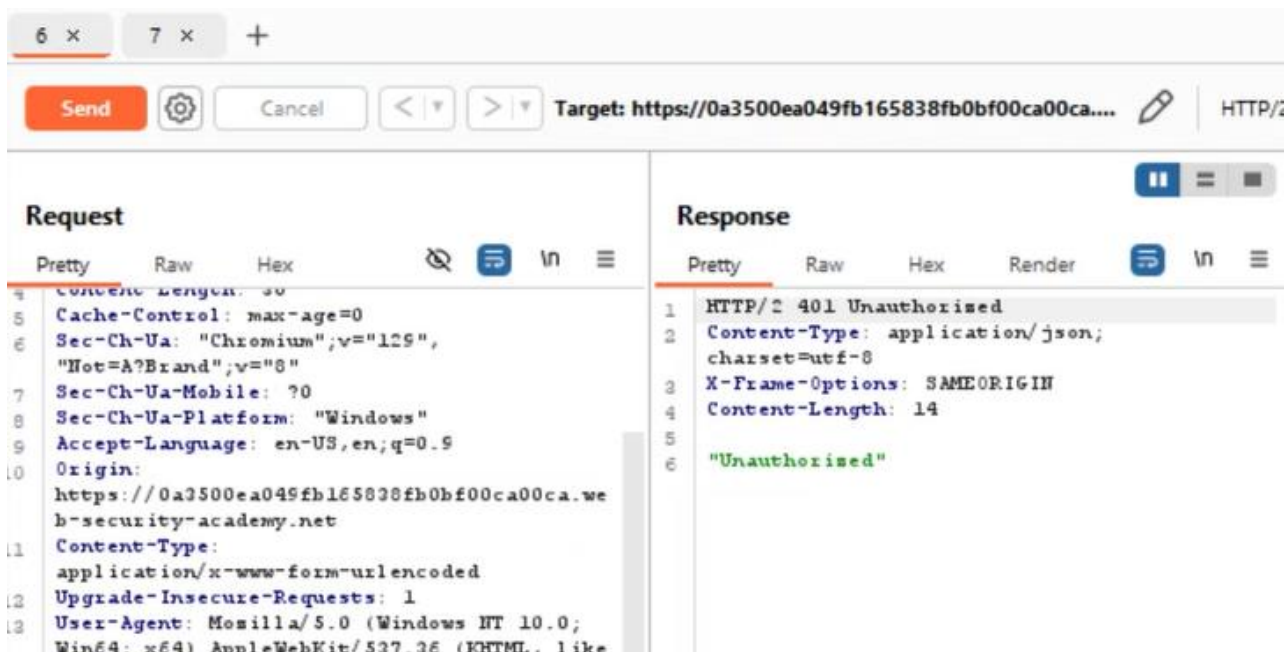
Burp Suite Response:

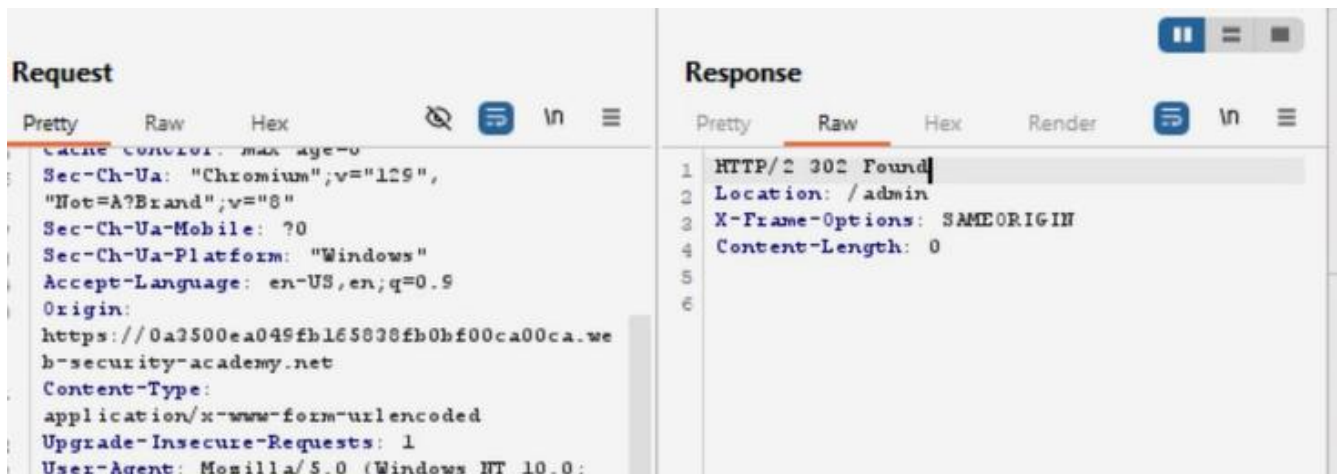
```
HTTP/2 200 Found
Location: /admin
X-Frame-Options: DENY
Content-Length: 0
```

Bước 4: Copy cookie session value

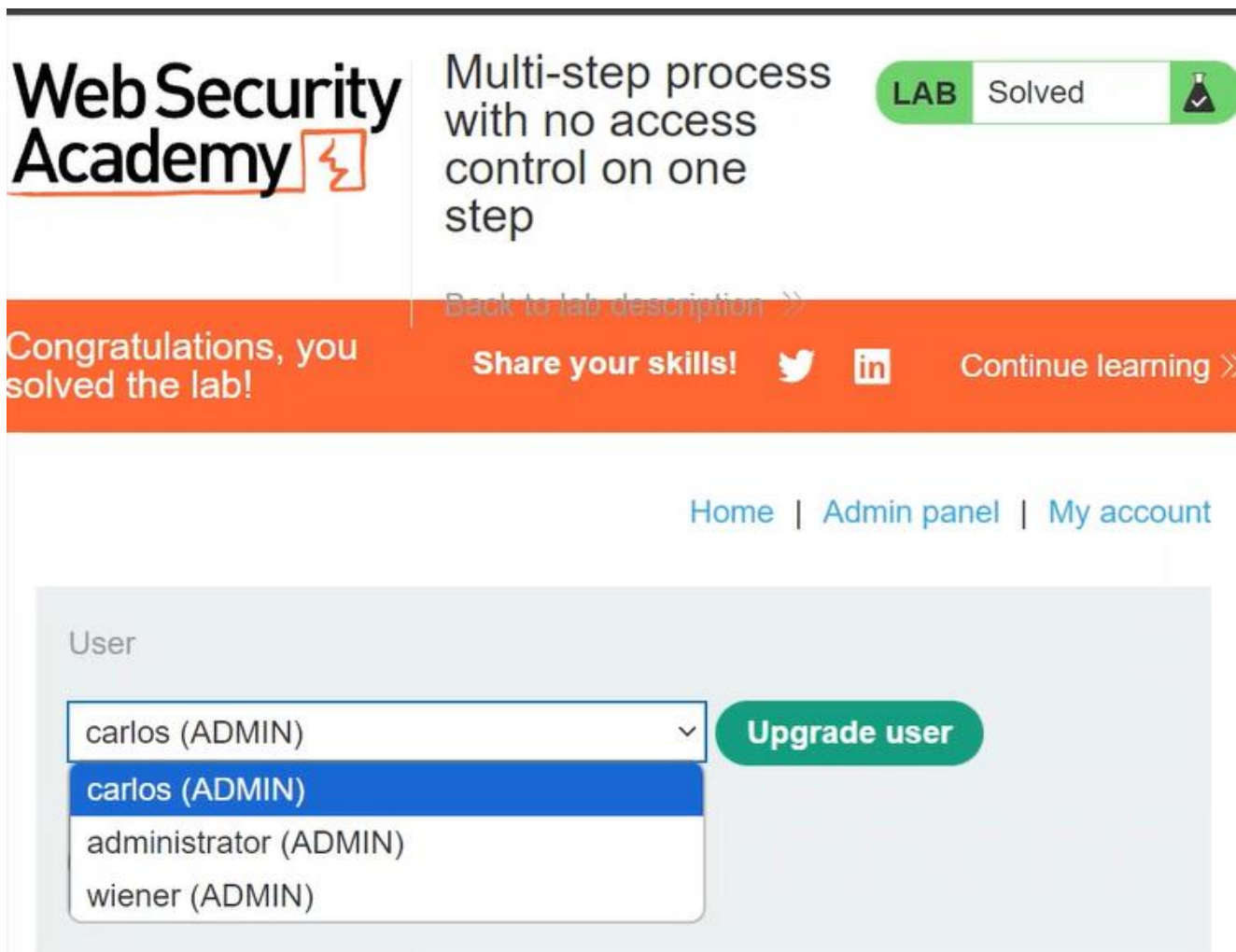


Bước 5: Chỉnh sửa cookie session value trong cả 2 gói tin trong phần repeat thành giá trị vừa copy và đổi username từ carlos thành wiener sau đó gửi lần lượt 2 gói tin đi, ta được kết quả



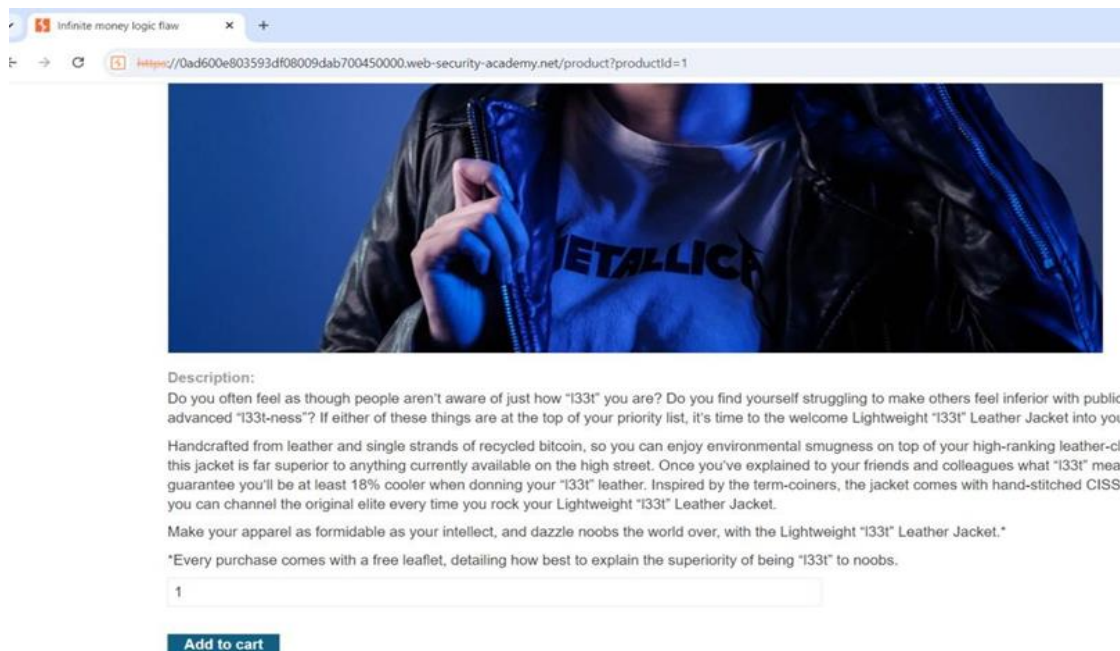


Bước 6: Kiểm tra thấy wiener đã có quyền Admin



<https://portswigger.net/web-security/logic-flaws/examples/lab-logic-flaws-infinite-money>

Truy cập vào trang, tìm kiếm món hàng cần mua là Lightweight “I33t” Leather Jacket.



- Chọn “Add to cart” để thêm vào giỏ hàng
- Ấn vào giỏ hàng để đến trang mua

Store credit:
\$100.00

Cart

Name	Price	Quantity
Lightweight "I33t" Leather Jacket	\$1337.00	- 1 + Remove

Coupon:

Add coupon

Apply

Total: \$1337.00

Place order

- Tại ô Add coupon, ta có thể nhập coupon để giảm giá, tuy nhiên khi ta hủy đơn thì số tiền mua (bao gồm tiền coupon) sẽ được thêm. Khi đó số dư tài khoản sẽ tăng, ta sẽ lợi

dùng điểm này để tăng số dư đến khi vượt hơn 1337 dollar

- Thực hiện Sign up để được một coupon “SIGNUP30” và mua một vật phẩm dưới 100 dollar (mua sticket giá 10 dollar), áp dụng coupon và thực hiện mua, khi đó trang hóa đơn sẽ hiện ra, copy dòng mã dưới “code” .
- Vào My Account -> Paste mã vào Gift cards -> Redeem thì thấy số tiền sẽ tăng lên
- Thực hiện bắt gói tin POST /gift-card sẽ thấy được mã gift-card mà mình đã dùng

No	URL	Method	Status	Size	Type	Info
182	https://0ad600e803593df080...	GET	200	7258	XML	svg
208	https://0ad600e803593df080...	GET	101	147		
209	https://0ad600e803593df080...	GET	200	4422	HTML	Infin
210	https://0ad600e803593df080...	GET	101	147		
211	https://0ad600e803593df080...	POST	302	91		
212	https://0ad600e803593df080...	GET	200	4423	HTML	Infin
213	https://0ad600e803593df080...	GET	101	147		

Request

```

Content-Length: 30
Cache-Control: max-age=0
Sec-Ch-Ua: "Not;A=Brand";v="24", "Chromium";v="128"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
Origin: https://0ad600e803593df08009dab700450000.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.6613.120 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0ad600e803593df08009dab700450000.web-security-academy.net/my-account?id=wiener
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
csrf=pQ9HxUnshZYlt340ksYp46AGjbYRiOn7&gift-card=4vRkb2lP3d

```

Response

```

1 HTTP/2 302 Found
2 Location: /my-account
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0
5
6

```

- Tại ứng dụng Burp Suite chọn Setting, tìm kiếm Session -> Tại “session handling rules” chọn Add
- Bên tab Scope chọn “Include all URLs”
- Bên tab Details chọn Add -> Run a macro
- Chọn Add, chọn 5 gói tin “POST /cart; POST /cart/coupon; POST /cart/checkout; GET /cart/order-confirmation?order-confirmed=true; POST /gift-card” -> OK
- Chọn gói tin “GET /cart/order-confirmation?order-confirmed=true” -> Configure items -> Add; tại Parameter name thì nhập tên -> Tìm mã code ở dưới phần HTML

-> OK

- Chọn gói tin “POST /gift-card” -> Configure items -> Đổi mục thứ 2 thành “Derive from prior responsibility” -> OK
- Nhấn OK đến khi quay lại giao diện chính
- Tại gói tin GET /my-account, thực hiện ấn chuột phải, chọn “Send to Intruder”
- Qua cửa sổ Intruder, phần Payload -> Payload type chọn “Null payloads”; phần Payload Setting nhập khoảng 415 payloads

Positions
Payloads
Resource pool
Settings

Payload sets

You can define one or more payload sets. The number of payload sets depends on the attack type defined in the Positions tab. Variou

Payload set: 1
Payload count: 415

Payload type: Null payloads
Request count: 0

Payload settings [Null payloads]

This payload type generates payloads whose value is an empty string. With no payload markers configured, this can be used to repeat

☒ Generate 415 payloads
☐ Continue indefinitely

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add
Edit
Remove
Up
Down

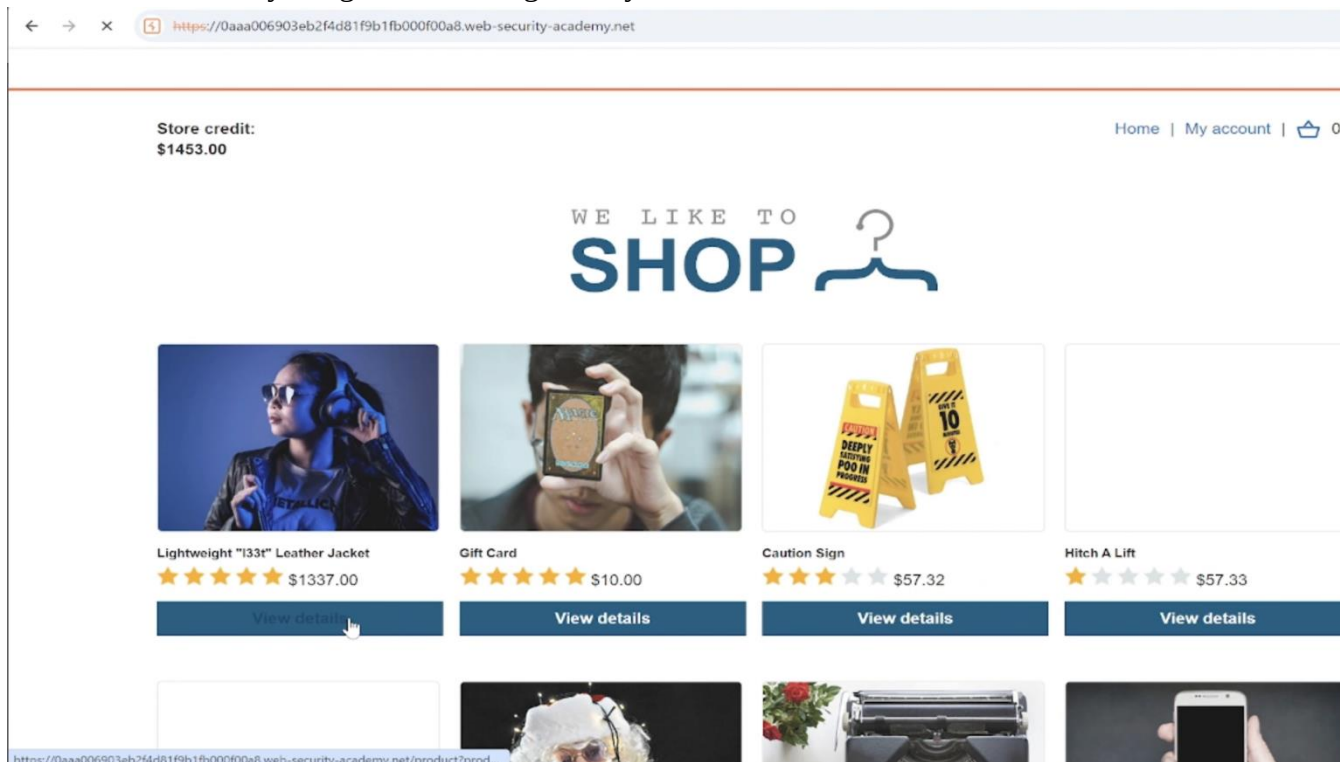
Enabled	Rule

Payload encoding

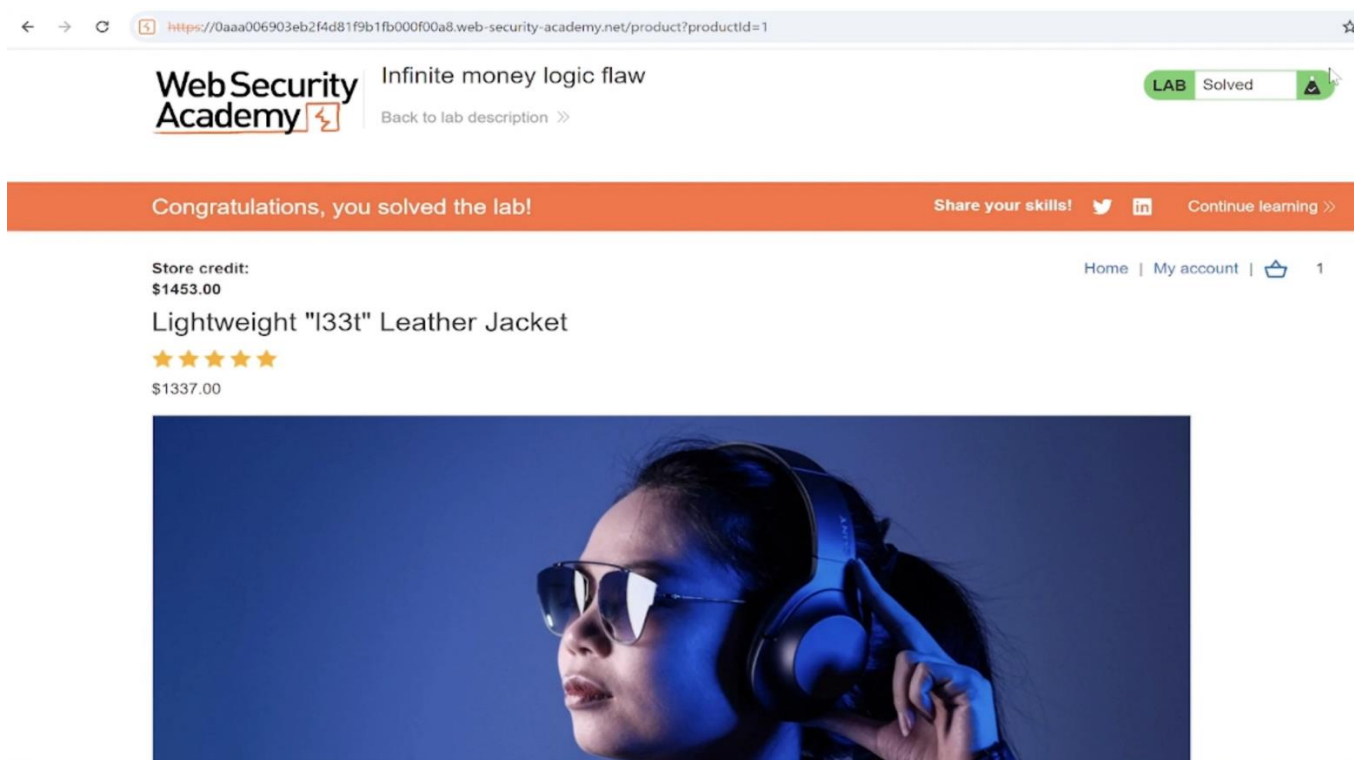
This setting can be used to URL-encode selected characters within the final payload, for safe transmission within HTTP requests.

☒ URL-encode these characters: = < > ? + & % " ' : [] \ ^ ` #

- Tại “Resource pool” chọn “Create new resource pool”; ô Maximum nhập 1
- Chạy “Start Attack”
- Sau khi chạy xong, load lại trang và thấy tiền đã đủ để mua -> Hoàn thành



- Lúc này giá tiền hiện có đã nhiều hơn giá tiền của món đồ cần mua, ta sẽ thực hiện mua hàng và hoàn thành lab.



<https://portswigger.net/web-security/prototype-pollution/client-side/browser-apis/lab-prototype-pollution-client-side-prototype-pollution-via-browser-apis>

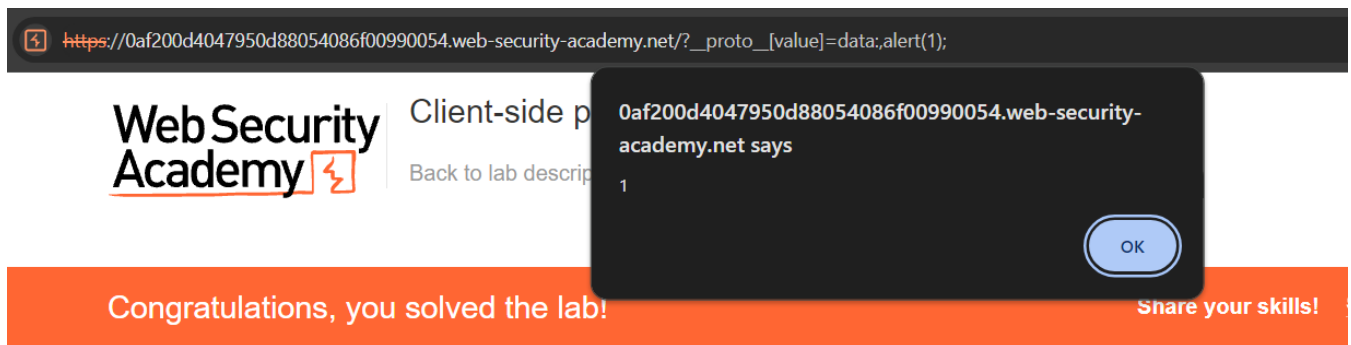
Mô tả: Prototype Pollution đây là một lỗ hổng xảy ra khi kẻ tấn công có thể sửa đổi các thuộc tính của prototype object (đối tượng mẫu) trong JavaScript. Điều này có thể dẫn đến việc thay đổi hành vi của toàn bộ đối tượng trong ứng dụng web, gây ra các hậu quả bảo mật như **XSS (Cross-Site Scripting)**

Tóm tắt:

1. **Chèn thuộc tính vào Object.prototype:** Kẻ tấn công chèn thuộc tính vào prototype thông qua URL hoặc các điểm nhập liệu khác của ứng dụng, như sau:
2. `/?__proto__[x-username]=<img/src/onerror=alert(1)>`
3. **Gán giá trị ô nhiễm cho đối tượng:** Trong ví dụ này, thuộc tính x-username được thêm vào và thừa kế bởi các đối tượng mới. Khi ứng dụng sử dụng thuộc tính này mà không kiểm tra đúng cách, mã JavaScript chèn vào có thể được thực thi trong ngữ cảnh trình duyệt của người dùng, gây ra DOM XSS.
4. **Lỗ hổng qua fetch():** Một ví dụ khác là khi ứng dụng sử dụng `fetch()` để gửi yêu cầu HTTP với các tùy chọn như phương thức và tiêu đề. Kẻ tấn công có thể ô nhiễm prototype của đối tượng chứa các tùy chọn này, dẫn đến thay đổi trong yêu cầu hoặc phản hồi, từ đó dẫn đến việc thực thi mã JavaScript.

Các bước:

Thêm dòng `/?__proto__[value]=data:;,alert(1)` vào sau



Cách khắc phục:

- Lập trình viên cần đảm bảo không cho phép người dùng thao tác trực tiếp trên các prototype.
- Cập nhật các thư viện JavaScript nếu chúng có lỗ hổng bảo mật liên quan đến Prototype Pollution.
- Thực hiện kiểm tra và lọc các đầu vào của người dùng một cách cẩn thận để ngăn chặn các lỗ hổng như vậy.